M. S. RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU, Belagavi)

Bengaluru – 560 054

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

QUANTUM PROJECT

Quantum Circuit Noise and Error Mitigation

Subject: Quantum Computing

Course Code: 23CYE732

Academic Year 2026 – 2027

VII Semester

Team Members

Name	USN
G Srikaanth	1MS23CY015
Devansh Sharma	1MS23CY014

Under the guidance of

Name: Dr. Sajni G

Designation: Professor

QUANTUM COMPUTING – GROUP RESEARCH ASSIGNMENT

Final Report

Group 20

Noise and Error Mitigation

Implemented with Qiskit + Qiskit Aer (Python)

Team Details

#	Student Name	Roll / Reg. No.	Role / Main Contribution
1	G Srikaanth	1MS23CY015	Project lead (about 70% of the work): Track A testbed and mitigation methods, VQC, all experiment runs, analysis and report writing
2	Devansh Sharma	1MS23CY014	Team member (about 30% of the work): Iris preprocessing and LR/SVM baselines, literature review, testing and report review

Course / Section	Quantum Computing (23CYE732), VII Semester, Department of Computer Science and Engineering (Cyber Security), M. S. Ramaiah Institute of Technology, Academic Year 2026–2027
Faculty Guide	Dr. Sajni G, Professor
Date of Submission	11 October 2026
Report (PDF) link	https://quantum-error-mitigation-bench.vercel.app/Group20_Quantum_Report.pdf
Code repository / Colab link	https://github.com/srikantlose/quantum-error-mitigation-bench (public) Interactive results dashboard and study guide: https://quantum-error-mitigation-bench.vercel.app

Assigned Group Brief (pre-filled – do not edit)

This table reproduces your group's assignment from the consolidated assignment sheet. Your report must address every row.

Research problem	Noise and Error Mitigation
Detailed problem statement	Investigate whether a selected error-mitigation method improves the output quality of a noisy quantum circuit.
Research objective	Test a simple error-mitigation technique.
Dataset required	Iris binary or small expectation-value circuit
Qubits	2,4,6
Circuit depth / algorithm parameter	Depth 2,4
Noise levels	Ideal, low, moderate
Common run settings	1024; 5 seeds
Baseline / comparison	No mitigation baseline
Metrics to observe	Accuracy/fidelity before and after mitigation, overhead
Expected research analysis	Effectiveness and cost of mitigation
Fixed parameters (assignment document)	Qubits: 2, 4; Noise: No noise, low noise, medium noise; Shots: 1024.
Experimental tasks	1. Select a circuit. 2. Establish a noiseless reference. 3. Add noise. 4. Apply a suitable error-mitigation technique if supported. 5. Compare results before and after mitigation.
Expected results	Error comparison, output distribution analysis, circuit overhead, and a discussion of mitigation limitations.

Common settings for all groups

Item	Standard requirement
Environment	Python 3.x, Qiskit, Qiskit Aer, NumPy, Pandas, Matplotlib, Scikit-learn
Data split	Stratified 80:20 train-test split; same split for classical and quantum models; fit scaling/PCA on training data only
Repeated runs	5 random seeds where feasible; report mean ± standard deviation
Shots	1024 per circuit execution
N0 – Ideal	No noise
N1 – Low noise	0.1% single-qubit, 0.5% two-qubit, 1% readout error
N2 – Moderate noise	1% single-qubit, 2% two-qubit, 3% readout error
N3 – High (optional)	3%–5% gate error and higher readout error (stress test)
Noise model	Depolarizing error + readout error (thermal relaxation optional if documented)
Minimum experiments	At least 6 controlled experiments, varying at least two parameters
Reporting	Tables/plots, limitations, research gap, fair comparison; do not claim quantum advantage without evidence

1. Problem Understanding and Objectives (2 marks)

1.1 Problem statement

Current noisy intermediate-scale quantum (NISQ) processors make mistakes in both their gates and their measurements. These errors bias the expectation values a circuit returns, usually shrinking them toward zero, and the bias grows with the number of qubits and the circuit depth. Full quantum error correction would remove the bias, but it needs many physical qubits per logical qubit and hardware that does not yet exist at useful scale [1]. Quantum error mitigation is the practical alternative for today's devices: it runs extra circuits and post-processes their results classically to remove bias from averaged quantities, without extra qubits [13]. Mitigation is not free. It costs extra circuits, shots and circuit depth, it can amplify statistical noise, and each method targets only part of the noise. This project asks whether two widely used methods, readout error mitigation (REM) and zero-noise extrapolation (ZNE), and their combination actually improve the output of a noisy circuit; what each costs; and whether the improvement carries through to a downstream quantum machine-learning task. We study this in a controlled Qiskit Aer simulation with the course-standard noise levels, on a parameterized circuit (2, 4 and 6 qubits; depth 2 and 4) and on a variational quantum classifier (VQC) for the Iris binary task, compared against Logistic Regression and SVM. The question matters because every near-term quantum algorithm depends on reading accurate expectation values from noisy hardware, and choosing a mitigation method means trading accuracy against cost.

1.2 Objectives

O1. Build a controlled, reproducible noisy-simulation testbed in Qiskit Aer with the course noise levels N0–N3 (2, 4 and 6 qubits; depth 2 and 4; 1024 shots; 5 seeds per condition).

O2. Implement REM (full and tensored) and ZNE (global folding with Richardson extrapolation), alone and combined, and measure the change in expectation-value error, Hellinger fidelity and success probability against the no-mitigation baseline.

O3. Measure each method's cost (circuits, shots, transpiled depth, two-qubit gates) and test whether the gains carry into a VQC on Iris binary, compared with Logistic Regression and SVM on identical data splits.

O4. Propose two lower-cost variants, tensored REM and variance-optimal ZNE shot allocation, and test both with data.

1.3 Research questions / hypotheses

RQ1. Do REM, ZNE and their combination reduce expectation-value error and improve the output distribution of a noisy circuit across noise levels N0–N3, 2–6 qubits and depth 2–4?

RQ2. What does each method cost in circuits, shots, depth and two-qubit gates, how does that scale with qubit count, and can cheaper variants (tensored REM, optimal shot allocation) keep the benefit?

RQ3. Do expectation-value improvements translate into better VQC classification metrics on Iris binary, and how does the VQC compare with Logistic Regression and SVM on identical inputs?

Ten hypotheses (H1–H10) were stated before the runs and are evaluated against the data in Table 5 (Section 6.2).

2. Literature Review (3 marks)

#	Authors, Year	Title / Venue / DOI	Method & dataset	Key finding and limitation
1	K. Temme, S. Bravyi, J. M. Gambetta, 2017 [2]	Error mitigation for short-depth quantum circuits. Phys. Rev. Lett. 119, 180509. DOI: 10.1103/PhysRevLett.119.180509	Theory: two schemes for short-depth circuits, extrapolation to the zero-noise limit (Richardson's deferred approach to the limit) and quasi-probability resampling of randomized circuits.	Expectation values can be corrected without extra qubits. Limitation (stated by the authors): the circuit size these schemes can handle is limited by the rate at which errors are introduced.

#	Authors, Year	Title / Venue / DOI	Method & dataset	Key finding and limitation
2	A. Kandala et al., 2019 [5]	Error mitigation extends the computational reach of a noisy quantum processor. Nature 567, 491–495. DOI: 10.1038/s41586-019-1040-7	Superconducting processor: extrapolation from a collection of experiments at varied noise; single- and two-qubit experiments, then variational optimization of Hamiltonians for chemistry and magnetism.	Suppressing incoherent errors recovers otherwise inaccessible accuracy with no hardware modifications. Limitation: demonstrated on chemistry and magnetism problems; not applied to a classification task.
3	T. Giurgica-Tiron, Y. Hindy, R. LaRose, A. Mari, W. J. Zeng, 2020 [6]	Digital zero noise extrapolation for quantum error mitigation. IEEE QCE 2020, 306–316. DOI: 10.1109/QCE49297.2020.00045	Introduces unitary folding $U(U^{\dagger}U)^{k}$ and parameterized noise scaling (gate-level access only) and compares extrapolation methods, including an adaptive one.	Benchmarks show 18× to 24× error reduction over unmitigated circuits. Limitation (for our setting): folding assumes the added gates scale the noise proportionally, and extrapolation still amplifies shot noise.
4	S. Bravyi, S. Sheldon, A. Kandala, D. C. McKay, J. M. Gambetta, 2021 [7]	Mitigating measurement errors in multiqubit experiments. Phys. Rev. A 103, 042605. DOI: 10.1103/PhysRevA.103.042605	IBM Quantum hardware: tensor-product and correlated Markovian readout-noise models with rates extracted from calibration data; graph states with $n \leq 12$ and 20-qubit random-Clifford states.	Measurement errors can be mitigated by classical post-processing with the inverse noise matrix. Limitation: overhead grows exponentially in εn (ε the maximum error rate); the tensor-product model ignores correlations.
5	M. Schuld, R. Sweke, J. J. Meyer, 2021 [15]	Effect of data encoding on the expressive power of variational quantum-machine-learning models. Phys. Rev. A 103, 032430. DOI: 10.1103/PhysRevA.103.032430	Theory: a quantum model written as a partial Fourier series in the data, with frequencies set by the encoding gates.	Repeating simple encoding gates gives richer frequency spectra; a single ry(x) encoding gives only frequencies {−1, 0, +1}. Limitation (for our setting): concerns expressivity, not noise or mitigation.
6	S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, P. J. Coles, 2021 [16]	Noise-induced barren plateaus in variational quantum algorithms. Nat. Commun. 12, 6961. DOI: 10.1038/s41467-021-27045-6	Theory: proof for a generic ansatz (including QAOA and UCC) under local Pauli noise, with numerical heuristics for QAOA on a realistic noise model.	Gradients vanish exponentially in qubit count when depth grows linearly with it, so noise alone can block training. Limitation (for our setting): concerns training under noise, whereas we study noise and mitigation at inference time.
7	R. Takagi, S. Endo, S. Minagawa, M. Gu, 2022 [11]	Fundamental limits of quantum error mitigation. npj Quantum Inf. 8, 114. DOI: 10.1038/s41534-022-00618-z	Theory: universal bounds on how far mitigation can reduce error as a function of sampling overhead.	For local depolarizing noise in layered circuits the sampling overhead grows exponentially with depth; PEC is optimal for local dephasing noise. Limitation (for our setting): theoretical bounds rather than measured performance of specific methods.
8	Z. Cai et al., 2023 [13]	Quantum error mitigation. Rev. Mod. Phys. 95, 045005. DOI: 10.1103/RevModPhys.95.045005	Review of proposed mitigation methods, their in-principle efficacy and hardware demonstrations.	Identifies commonalities and limitations, and notes the method should be chosen according to the dominant type of noise. Limitation (for our setting): a review, not a controlled benchmark.

2.1 Research gap

The reviewed papers study readout mitigation [7], [8] and extrapolation-based mitigation [2], [5], [6] mostly separately, often at hardware scale, and report gains as estimator bias rather than in cost-normalized terms. Theoretical work bounds the cost of mitigation [11], [12] but does not compare specific methods side by side, and QML work [14]–[16] rarely asks whether mitigation's effect on expectation values survives into classification metrics measured against classical baselines on identical inputs. This project addresses that gap at small, fully controlled scale: one paired protocol in which every method post-processes the same shots,

across a physics-style testbed and a QML task, reporting error, cost and their trade-off together, and testing two lower-cost variants: tensored REM, following the direction of [7], and variance-optimal ZNE shot allocation, which to our knowledge is not addressed directly in the papers reviewed.

3. Algorithm / Circuit Design (3 marks)

3.1 Dataset and preprocessing

Dataset & source (URL / sklearn loader)	Track B: Iris dataset (R. A. Fisher, 1936; UCI Machine Learning Repository, DOI 10.24432/C56C76), loaded with `sklearn.datasets.load_iris`. Binary task: versicolor (+1) vs. virginica (−1). Track A uses no dataset: random parameterized parity circuits with an exact statevector reference (the "small expectation-value circuit" option).
Samples / features / classes	100 samples (50 versicolor, 50 virginica), 4 features (sepal length and width, petal length and width), 2 classes. Setosa is excluded because it is linearly separable from the other two classes, which would make the task trivial.
Feature reduction (PCA / selection) and final feature count	n = 2 qubits: PCA to 2 components, fitted on the training split. n = 4 qubits: all 4 features, no reduction. One feature per qubit.
Scaling method (fit on training data only)	StandardScaler, then PCA (n = 2 only), then MinMaxScaler to [0, π]. Every step is fitted on the 80 training samples only; test samples are transformed and clipped to [0, π]. A unit test checks that refitting on train + test gives different parameters.
Train / test split (stratified 80:20), random seed	Stratified 80:20 (80 train / 20 test) with `train_test_split(..., stratify=y, random_state=derive_seed("qml_split", seed))` for seeds 0–4. The VQC, Logistic Regression and SVM use the identical index arrays.

3.2 Encoding, ansatz / algorithm

Track A circuit

Each instance has n qubits and L layers. A layer applies ry(θ) then rz(ϕ) to every qubit, then the linear CNOT chain cx(0,1), cx(1,2), …, cx(n−2, n−1), giving 2nL single-qubit gates and (n−1)L CNOTs. Angles are drawn uniformly from [0, 2π) from a seeded stream and redrawn until the exact parity satisfies |E_exact| ≥ 0.3 (median 2 draws, at most 35; selected |E_exact| from 0.311 to 0.872). This keeps the signal well above the shot-noise floor (σ ≈ 0.031 at 1024 shots), where relative error is defined. Random layered rotations with a CNOT chain give entangled states whose output is sensitive to noise on every qubit, and the gate set {ry, rz, cx} is exactly the set the noise model covers.

Measurement and observable

All qubits are measured in the computational basis. The observable is the global parity $\langle Z \otimes \ldots \otimes Z \rangle = \Sigma_x$ p(x)·(−1)^(number of 1s in x), which lies in [−1, 1]. A bit flip on any qubit flips its sign, so it is maximally sensitive to both gate and readout noise and makes the effect of mitigation easy to see. The exact reference E_exact comes from a statevector simulation; distribution metrics (Hellinger fidelity, TVD, success probability of the most likely ideal bitstring) compare against the exact noiseless distribution.

Readout error mitigation (REM)

For each of the 2^n basis states a calibration circuit prepares it with ry(π) gates (staying inside the noisy gate basis) and measures it with 1024 shots. Column j of the assignment matrix A is the measured distribution when state j was prepared, A[i, j] = P(measure i | prepared j). The mitigated quasi-probabilities are $q = A^{-1}\tilde{p}$ (least squares if cond(A) > 10^8, which happened 0 times). The parity is computed from the unprojected q, which keeps it linear and unbiased; q is projected onto the probability simplex [17] only for Hellinger fidelity and TVD. Tensored REM [7] measures only the all-|0⟩ and all-|1⟩ states, builds a 2×2 matrix per qubit and forms $A = A_{(n-1)} \otimes \ldots \otimes A_0$, assuming independent readout errors: 3 circuits per estimate at any n, instead of $1 + 2^n$.

Zero-noise extrapolation (ZNE)

Global unitary folding [6] replaces the circuit U with $U(U^\dagger U)^k$ for scale factor $\lambda = 2k + 1 \in \{1, 3, 5\}$: the ideal output is unchanged, but the circuit runs λ times the gates and roughly λ times the noise. $U^\dagger$ negates every rotation angle and cx is its own inverse, so folded circuits stay in the noisy basis. Circuits are transpiled at optimization_level 0 so the $U^\dagger U$ pairs are not cancelled. Richardson extrapolation [2] gives $E_0 = \Sigma_i \gamma_i E(\lambda_i)$ with $\gamma = (15/8, -5/4, 3/8)$; linear and exponential [4] fits are recorded as diagnostics. Measurements are not folded, so ZNE alone cannot remove readout error; ZNE+REM applies REM to the counts at each λ before extrapolating.

Track B: variational quantum classifier

Each scaled feature $x_k \in [0, \pi]$ is encoded as $ry(x_k)$ on qubit k (angle encoding), so $\langle Z_k \rangle = \cos x_k$ before the ansatz; $[0, \pi]$ maps the minimum to $|0\rangle$ and the maximum to $|1\rangle$ without wrap-around. The ansatz is the same ry–rz–CNOT-chain block as Track A with 2nL trainable weights (8, 16, 16 and 32 for (n, L) = (2, 2), (2, 4), (4, 2), (4, 4)). The output is $f(x) = \langle Z^{\otimes n} \rangle \in [-1, 1]$ and the prediction is sign f(x), with 0 mapped to +1. Training minimizes the mean squared error against $y \in \{+1, -1\}$ with COBYLA (maxiter = max(300, 25 × parameters)) on the exact StatevectorEstimator, batching all 80 training samples per objective call. Training is noiseless on purpose: it isolates noise and mitigation to inference time and avoids the noise-induced vanishing gradients proved by Wang et al. [16]. A single encoding layer limits the model to Fourier frequencies {−1, 0, +1} per feature [15], a stated limitation. At test time each sample's circuit is run at every noise level with none, REM, ZNE and ZNE+REM (1024 shots per circuit), sharing one calibration across the 20 test samples.

3.3 Circuit diagram(s)

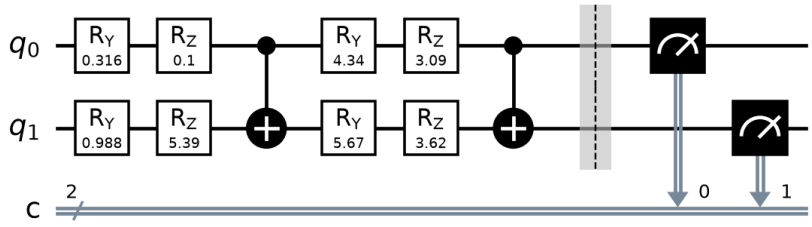


Figure 1: Track A, smallest configuration (n = 2, L = 2; the seed-0 instance actually run), with measurement.

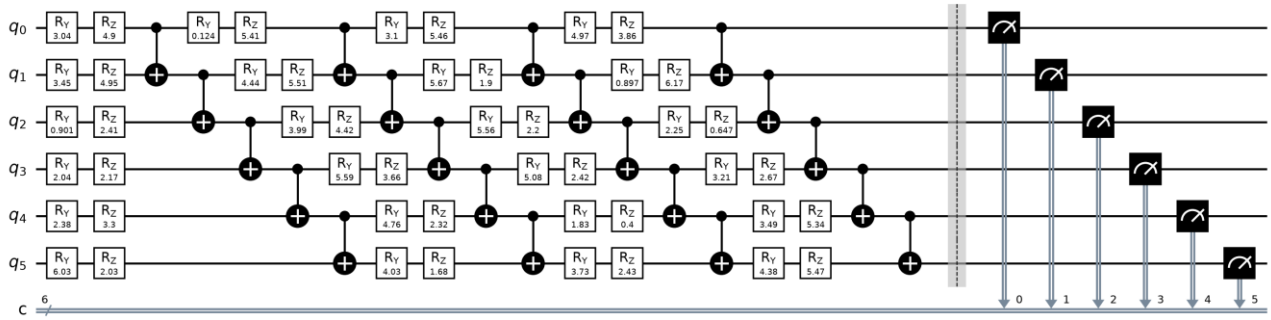


Figure 2: Track A, largest configuration (n = 6, L = 4; seed-0 instance). Each layer is ry and rz on every qubit, then a CNOT chain.

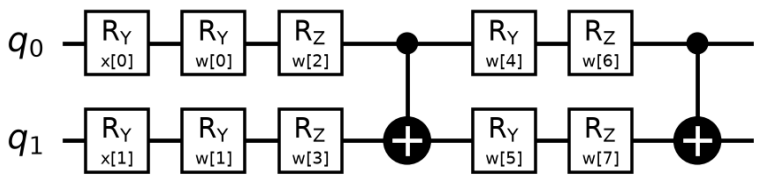


Figure 3: VQC, smallest configuration (n = 2, L = 2): angle encoding ry(x[k]), then the trainable ansatz with weights w[k].

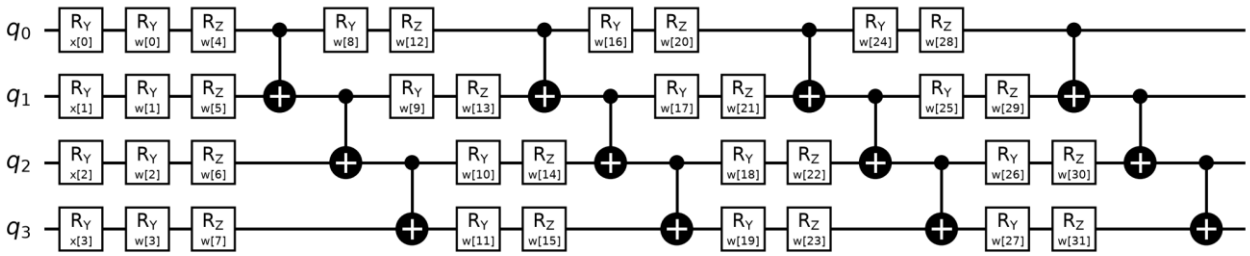


Figure 4: VQC, largest configuration (n = 4, L = 4): 4 features, 32 trainable weights.

D2. Global folding at λ = 3: U, then U†, then U again (n = 2, L = 1)

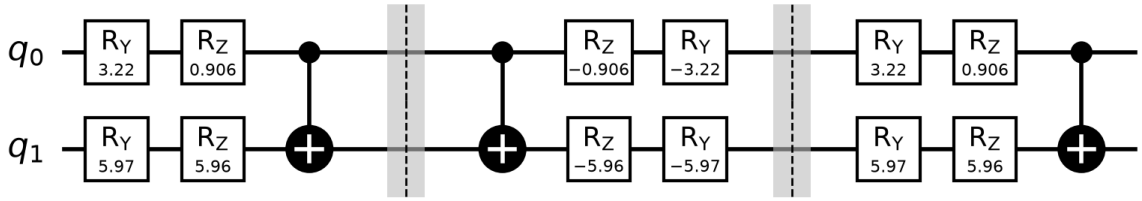


Figure 5: Global folding at λ = 3 (n = 2, L = 1): U, then U†, then U again. Same ideal output, three times the gates.

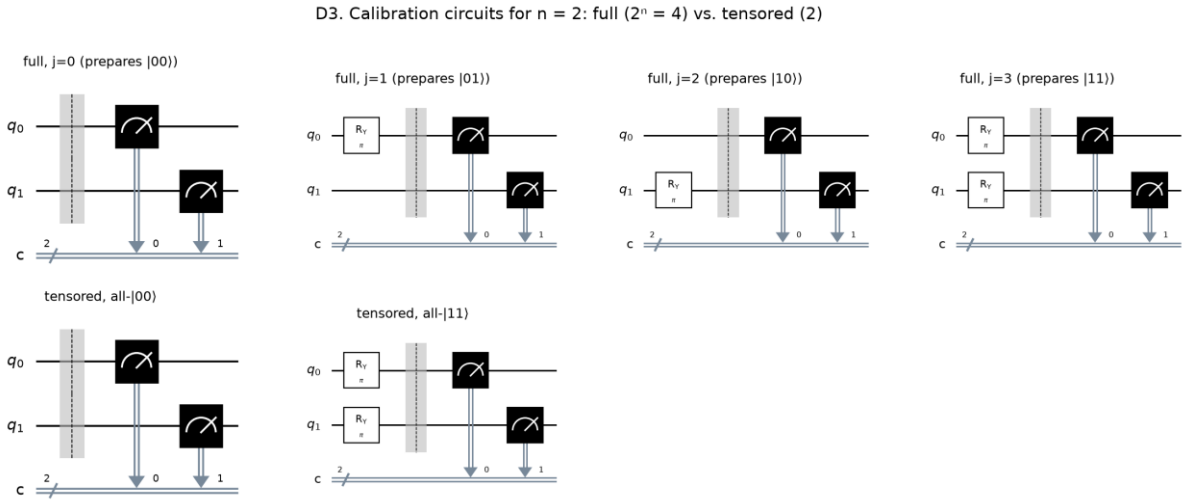


Figure 6: Calibration circuits for n = 2: full REM (2ⁿ = 4 circuits) vs. tensored REM (2 circuits at any n).

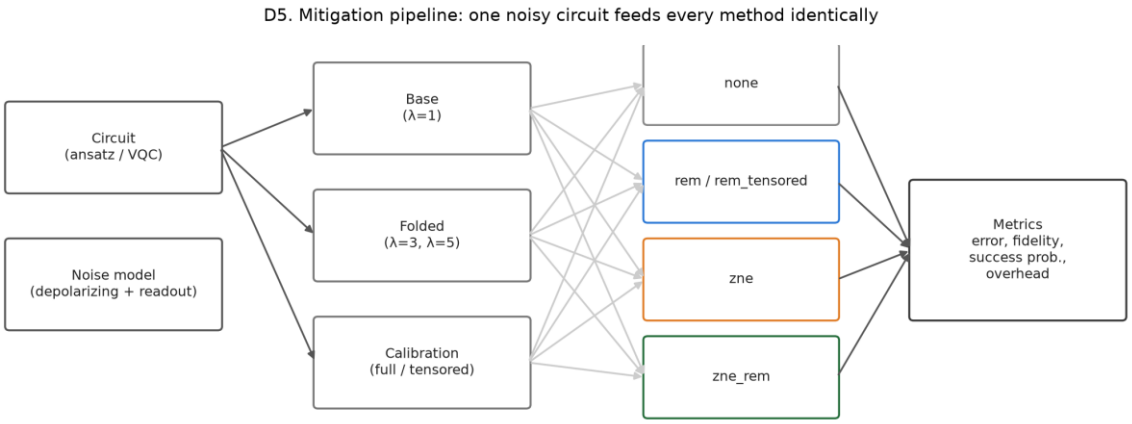


Figure 7: Mitigation pipeline: one circuit's base, folded and calibration runs feed every method from the same shots.

3.4 Parameter table

Parameter	Value(s) used	Justification
Number of qubits	Track A: 2, 4, 6 Track B: 2, 4	2 and 4 are the course standard; 6 is in the group brief and exposes full REM's 2^n calibration growth (65 circuits per estimate at n = 6).
Circuit depth / algorithm parameter	L = 2, 4 layers (ry–rz–CNOT chain) ZNE scale factors λ = 1, 3, 5	Course depths. λ = 1, 3, 5 are the smallest odd factors reachable by global folding and give a quadratic Richardson fit.
Optimizer & iterations	VQC: COBYLA, maxiter = max(300, 25 × parameters). (n, L) = (2, 2): maxiter 300, 244–300 evaluations used; (n, L) = (2, 4): maxiter 400, 400 evaluations used; (n, L) = (4, 2): maxiter 400, 400 evaluations used; (n, L) = (4, 4): maxiter 800, 800 evaluations used. Track A: no optimizer (fixed random angles)	Derivative-free and robust for 8–32 parameters; the iteration budget grows with the parameter count.
Shots	1024	Course standard; shot-noise σ ≈ 0.031 at E = 0. The shot-allocation experiment keeps the total 3 × 1024 = 3072 but splits it unevenly across λ.
Noise levels used	N0 ideal; N1 low (0.1% / 0.5% / 1%); N2 moderate (1% / 2% / 3%); N3 high (3% / 5% / 5%, stress test)	Course levels (single-qubit / two-qubit / readout). N3 probes where ZNE should strain and is reported separately.
Random seeds	5 seeds (0–4) per condition; every random number from derive_seed(*parts), a SHA-256 hash of named parts	Mean ± SD over 5 seeds as required; a rerun reproduces every output exactly except two wall-clock columns.

4. Qiskit Implementation (4 marks)

4.1 Environment and reproducibility

Item	Value
Python version	3.11.15 (uv-managed virtual environment)

Item	Value
Qiskit version	2.5.2
Qiskit Aer version	0.17.2
scikit-learn / NumPy version	scikit-learn 1.9.1 / NumPy 2.4.6 (also SciPy 1.17.1, pandas 3.0.6, Matplotlib 3.11.2)
Simulator / backend	Qiskit Aer (AerSimulator), method="density_matrix"; transpiled at optimization_level 0 to the basis {ry, rz, cx}
Random seeds (data split, model, simulator)	Seeds 0–4. Data split derive_seed("qml_split", seed); PCA derive_seed("qml_pca", seed); VQC initial weights derive_seed("qml_init", n, L, seed); LR/SVM derive_seed("qml_lr" / "qml_svm", n, seed). Track A circuit angles derive_seed("circuit", n, L, seed); simulator and transpiler derive_seed("sim" / "transpile", n, L, noise, seed, purpose) (Track B: "qml_sim" / "qml_transpile").
Noise-model parameters (N1 / N2 / N3)	N1: $p_1 = 0.001$, $p_2 = 0.005$, p_ro = 0.01 N2: $p_1 = 0.01$, $p_2 = 0.02$, p_ro = 0.03 N3: $p_1 = 0.03$, $p_2 = 0.05$, p_ro = 0.05 depolarizing_error(p_1, 1) on ry and rz; depolarizing_error(p_2, 2) on cx; symmetric ReadoutError with flip probability p_ro on every qubit
Hardware used (CPU / RAM / runtime)	AMD Ryzen 5 7600 (6 cores / 12 threads), 31.1 GB RAM as reported by Windows, Windows 11 Pro; CPU only. Runtime: Track A sweep 17.1 s (600 rows); Track B 494.1 s (360 rows, including VQC training); shot-allocation experiment 115.1 s; test suite 186 passed in 40.06s.

4.2 Code structure

The code is a Python package, `qem` (src/qem/), driven by command-line scripts. It runs end to end from a clean environment with `pip install -r requirements.txt` and `pip install -e .`. Repository (public): https://github.com/srikantlose/quantum-error-mitigation-bench.

- `config.py`: loads and validates config/experiment.yaml (noise levels, grid, seeds, shots); fixed settings such as optimization_level 0 are enforced.
- `seeds.py`: derive_seed(*parts), a SHA-256-based seed per purpose; every random number in the project comes from here.
- `circuits.py`: the Track A ansatz, instance selection, fold_global, and full and tensored calibration circuits.
- `observables.py`: parity, success probability and the exact statevector reference.
- `noise.py` and `execution.py`: NoiseModel construction and the Aer executor with the gate-basis guard.
- `mitigation/readout.py`, `mitigation/zne.py`, `mitigation/pipeline.py`: REM (full, tensored, simplex projection), Richardson/linear/exponential extrapolation, and the pipeline that feeds every method the same shots.
- `metrics.py` and `experiment.py`: per-run metrics and the Track A sweep (results/raw/runs.csv).
- `qml/` (data, vqc, classical, evaluate, runner): Track B preprocessing, VQC training, LR and SVM baselines and noisy evaluation.
- `improvement.py`: the variance-optimal ZNE shot-allocation experiment.
- `analysis.py` and `plotting.py`: summary tables, statistical tests and figures; scripts/build_report.py fills the report from these outputs so no result number is typed by hand.

The full pipeline, in order:

```
python scripts/run_sweep.py --config config/experiment.yaml          # Track A
python scripts/run_qml.py --config config/experiment.yaml            # Track B
python scripts/run_improvement.py --config config/experiment.yaml    # shot allocation
python scripts/analyze.py --config config/experiment.yaml            # tables and statistics
python scripts/make_plots.py --config config/experiment.yaml         # figures
python scripts/make_design_figures.py --config config/experiment.yaml
pytest -q                                                             # 186 tests
```

The 186 tests check, among other things, that folded circuits have the same statevector and exactly λ times the gates, the Richardson weights, that with readout-only noise REM recovers the exact value within 4σ where no mitigation does not, the overhead formulas, that preprocessing never sees test data, and that a rerun reproduces runs.csv exactly (apart from the two wall-clock columns) and every raw counts, circuit and calibration file byte for byte.

4.3 Noise model implementation

The noise model is built once per noise level in src/qem/noise.py and attached to an AerSimulator in src/qem/execution.py:

```
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, ReadoutError, depolarizing_error


def build_noise_model(
    level: NoiseLevelConfig, one_q_gates: Sequence[str], two_q_gates: Sequence[str]
) -> NoiseModel | None:
    if level.p1 == 0 and level.p2 == 0 and level.p_ro == 0:
        return None
    nm = NoiseModel()
    if level.p1 > 0:
        nm.add_all_qubit_quantum_error(depolarizing_error(level.p1, 1), list(one_q_gates))
    if level.p2 > 0:
        nm.add_all_qubit_quantum_error(depolarizing_error(level.p2, 2), list(two_q_gates))
    if level.p_ro > 0:
        # Row k gives P(measured | prepared k).
        nm.add_all_qubit_readout_error(ReadoutError([[1 - level.p_ro, level.p_ro], [level.p_ro, 1 -
level.p_ro]]))
    return nm

# src/qem/execution.py, Executor.__init__: one simulator per noise level
    for name, lvl in self._levels.items():
        nm = build_noise_model(lvl, cfg.noise.one_qubit_gates, cfg.noise.two_qubit_gates)
        if nm is None:
            self._sims[name] = AerSimulator(method=cfg.simulator.method)
        else:
            self._sims[name] = AerSimulator(method=cfg.simulator.method, noise_model=nm)

# Executor.run: transpile at optimization_level 0 to {ry, rz, cx}, then
result = sim.run(tcircs, shots=shots, seed_simulator=seed).result()
```

Gate errors are attached by gate name to every ry and rz (one-qubit) and cx (two-qubit) instruction, and the readout error is symmetric and identical on every qubit. Because noise is attached by name, any other gate would run noise-free, so the executor asserts before and after transpiling that every circuit contains only {ry, rz, cx, measure, barrier}. The exact noisy reference E_noisy_exact comes from a density-matrix run of the same circuit with no shots, which lets us split the unmitigated bias exactly into a gate part and a readout part.

4.4 Screenshots of working code and outputs

Environment, versions and tests. A terminal in the project's virtual environment printing the Python and library versions, then running the full test suite.

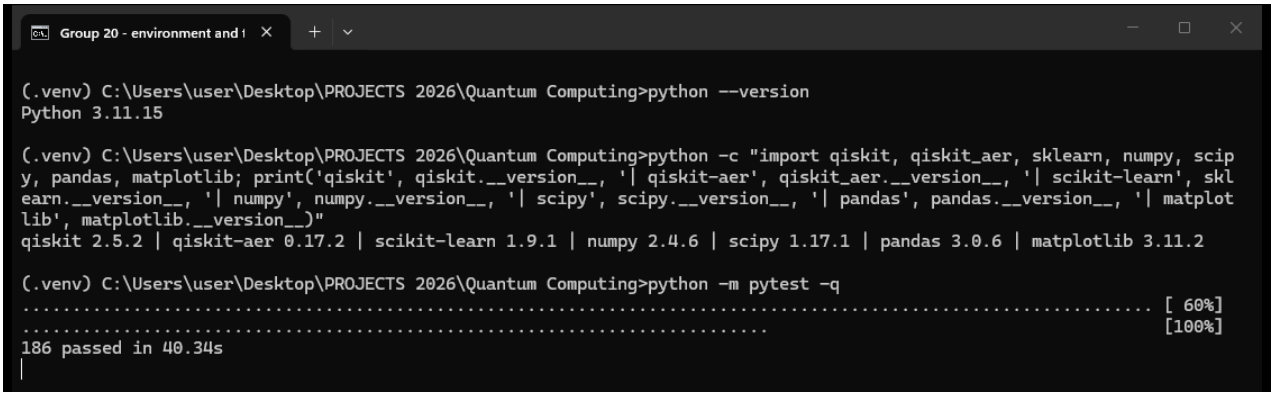


Figure 8: Terminal screenshot: Python 3.11.15, the library versions, and `pytest -q` *with all 186 tests passing.*

A successful run. The full Track A sweep (120 conditions, 600 result rows) rerun from scratch into a separate folder and compared with the committed results: the rerun is identical apart from the two wall-clock timing columns, which shows that the experiment is reproducible.

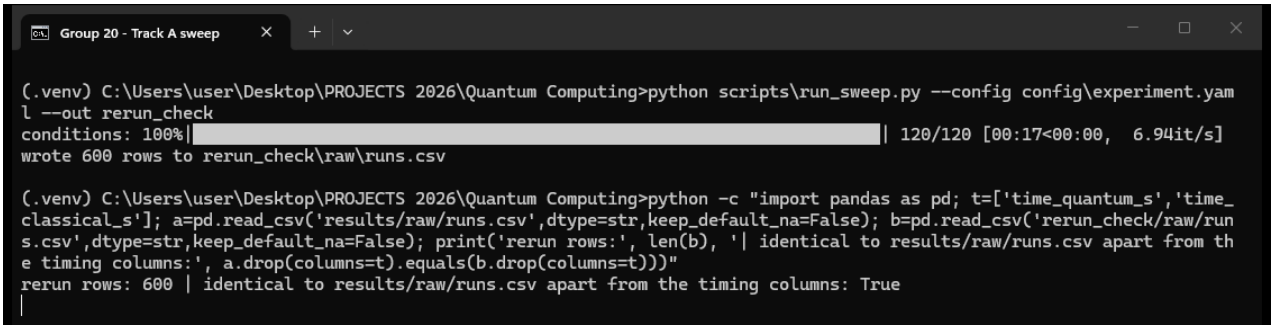


Figure 9: Terminal screenshot: the Track A sweep completing (120/120 conditions, 600 rows) and the reproducibility check returning True.

Code. The noise-model builder as it appears in the public repository (Section 4.3 discusses it).

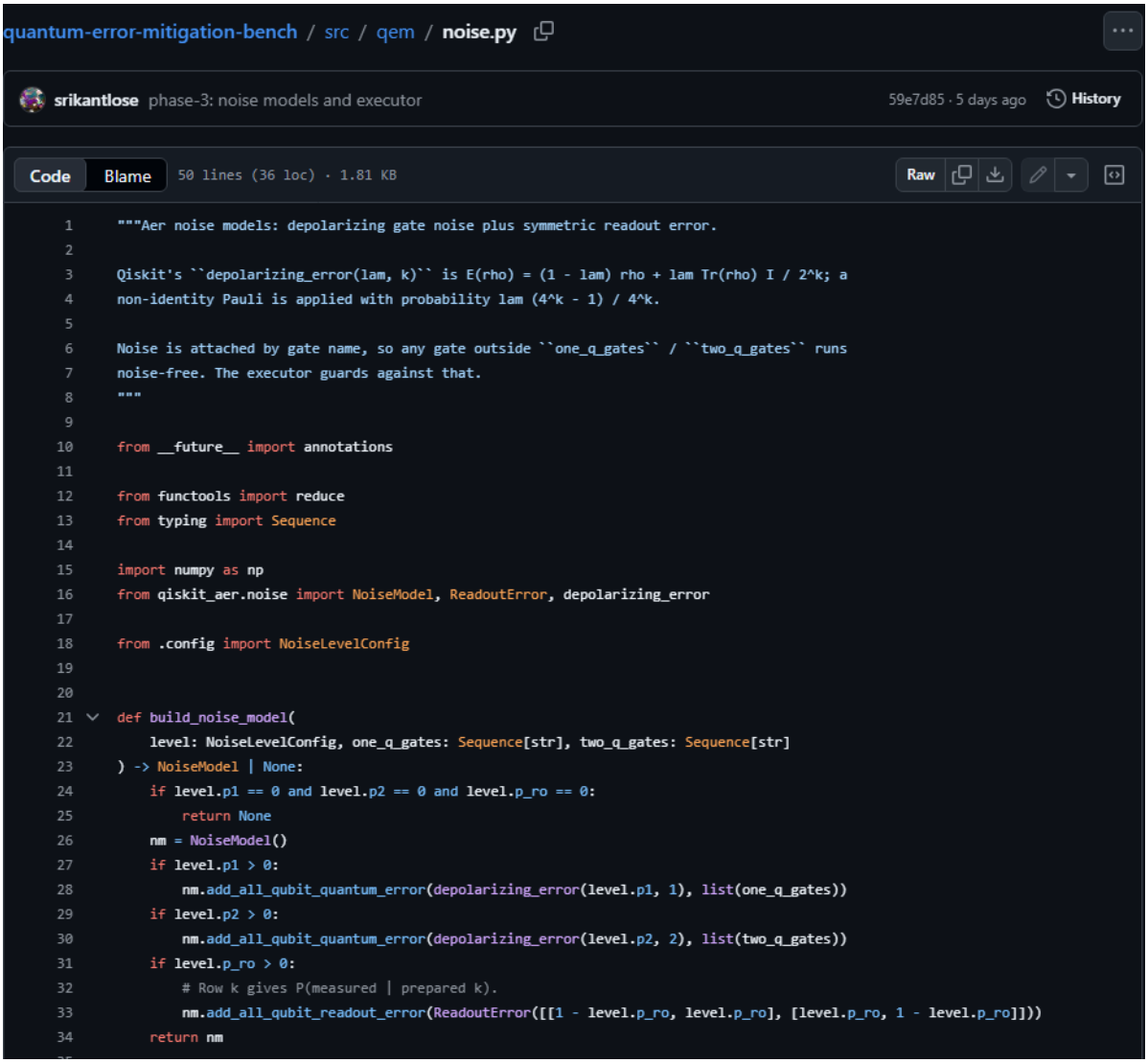


Figure 10: Code screenshot: build_noise_model in src/qem/noise.py on GitHub.

Sample output. The interactive results dashboard, built from the result files and deployed at https://quantum-error-mitigation-bench.vercel.app.

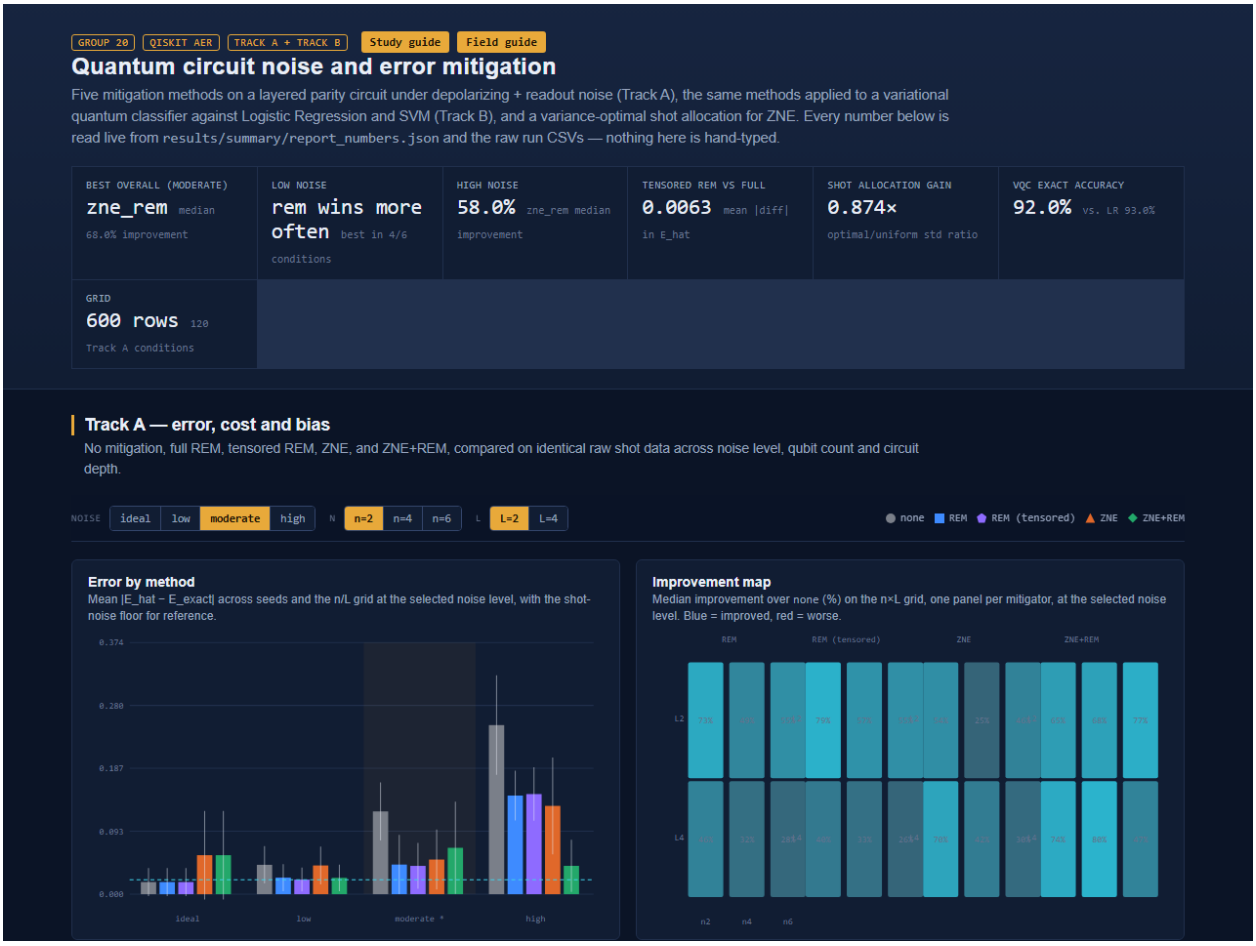


Figure 11: Screenshot of the deployed results dashboard: headline numbers, error by method and noise level, and the improvement map.

5. Experiments and Visualization (3 marks)

5.1 Experiment configuration

Table 1. Experiment configuration (the recommended matrix; all eight rows were run).

Exp.	Qubits	Depth / parameter	Noise	Shots	Notes (dataset, seeds, optimizer iterations)
E1	2	2	N0 – None	1024	Track A: 5 seeds × 5 methods. Track B: VQC on 2 PCA features, COBYLA maxiter 300 (244–300 evaluations used), 4 mitigation methods.
E2	2	4	N0 – None	1024	Track A as E1. Track B: VQC, COBYLA maxiter 400 (400 used).
E3	4	2	N0 – None	1024	Track A as E1. Track B: VQC on all 4 features, COBYLA maxiter 400 (400 used).
E4	4	4	N0 – None	1024	Track A as E1. Track B: VQC on all 4 features, COBYLA maxiter 800 (800 used).
E5	4	4	N1 – Low	1024	As E4 at N1 (0.1% / 0.5% / 1%); all methods; also the shot-allocation experiment (50 repetitions).
E6	4	4	N2 – Moderate	1024	As E4 at N2 (1% / 2% / 3%); all methods; also the shot-allocation experiment.

Exp.	Qubits	Depth / parameter	Noise	Shots	Notes (dataset, seeds, optimizer iterations)
E7	6	4	N0 – None	1024	Track A only (Track B uses 2 and 4 qubits); 5 seeds × 5 methods.
E8	6	4	N2 – Moderate	1024	Track A only; full REM needs 65 circuits per estimate here.

These eight rows are a subset of the full factorial design that was run. Track A covered every combination of n ∈ {2, 4, 6}, L ∈ {2, 4}, N0–N3 and 5 seeds (120 conditions, 600 method rows). Track B covered every combination of n ∈ {2, 4}, L ∈ {2, 4}, N0–N3 and 5 seeds with four mitigation methods, plus the exact VQC, Logistic Regression and SVM (360 rows). The shot-allocation experiment produced 120 rows at N1 and N2. Every circuit execution used 1024 shots, except the allocation experiment, which kept the total budget of 3 × 1024 = 3072 shots but split it unevenly across λ.

5.2 Results table

Table 2. Track A results for E1–E8, mean ± SD over 5 seeds. Primary metric: absolute parity error |E_hat − E_exact| after REM and after ZNE+REM. Secondary metric: Hellinger fidelity of the output distribution, before → after REM. Total gates and two-qubit gates: transpiled base circuit / summed over the three ZNE circuits (λ = 1, 3, 5). Depth: base circuit / deepest ZNE circuit (λ = 5). Baseline: no mitigation.

Exp.	Primary metric (mean ± SD)	Secondary metric	Total gates	Two-qubit gates	Depth	Baseline value
E1	REM: 0.018 ± 0.021 ZNE+REM: 0.058 ± 0.066	0.999 → 0.999	10 / 90	2 / 18	6 / 30	0.018 ± 0.021
E2	REM: 0.026 ± 0.021 ZNE+REM: 0.053 ± 0.041	0.999 → 0.999	20 / 180	4 / 36	12 / 60	0.026 ± 0.021
E3	REM: 0.016 ± 0.011 ZNE+REM: 0.051 ± 0.016	0.996 → 0.996	22 / 198	6 / 54	9 / 45	0.016 ± 0.011
E4	REM: 0.013 ± 0.009 ZNE+REM: 0.042 ± 0.042	0.997 → 0.997	44 / 396	12 / 108	17 / 85	0.013 ± 0.009
E5	REM: 0.040 ± 0.025 ZNE+REM: 0.080 ± 0.035	0.991 → 0.994	44 / 396	12 / 108	17 / 85	0.071 ± 0.027
E6	REM: 0.144 ± 0.028 ZNE+REM: 0.051 ± 0.021	0.952 → 0.968	44 / 396	12 / 108	17 / 85	0.209 ± 0.039
E7	REM: 0.027 ± 0.011 ZNE+REM: 0.049 ± 0.031	0.981 → 0.981	68 / 612	20 / 180	19 / 95	0.027 ± 0.011
E8	REM: 0.173 ± 0.056 ZNE+REM: 0.109 ± 0.049	0.908 → 0.927	68 / 612	20 / 180	19 / 95	0.240 ± 0.052

At N0 mitigation cannot help: REM is an exact no-op (largest change 0) and ZNE only adds variance; it was worse than no mitigation in 26/30 runs. At N1 (E5), REM roughly halves the error while ZNE does not help. At N2 (E6, E8) every method helps and ZNE+REM is best. Full tables for all 120 conditions are in results/summary/table_error.md and table_fidelity.md.

Table 3. Track B results: VQC on the 20-sample test set, mean ± SD over n ∈ {2, 4}, L ∈ {2, 4} and 5 seeds (20 runs per row; LR and SVM: 10 runs). Margin retention = mean(E_hat · sign E_exact) / mean |E_exact|.

Noise	Method	Accuracy	Mean |E_hat − E_exact|	Margin retention
ideal	none	0.920 ± 0.041	0.019 ± 0.003	0.997 ± 0.009
ideal	rem	0.920 ± 0.041	0.019 ± 0.003	0.997 ± 0.009
ideal	zne	0.922 ± 0.044	0.044 ± 0.008	0.994 ± 0.025
ideal	zne_rem	0.922 ± 0.044	0.044 ± 0.008	0.994 ± 0.025
low	none	0.920 ± 0.047	0.053 ± 0.024	0.907 ± 0.037

Noise	Method	Accuracy	Mean |E_hat − E_exact|	Margin retention
low	rem	0.917 ± 0.049	0.030 ± 0.010	0.966 ± 0.030
low	zne	0.907 ± 0.054	0.058 ± 0.009	0.942 ± 0.039
low	zne_rem	0.907 ± 0.054	0.053 ± 0.006	1.004 ± 0.043
moderate	none	0.917 ± 0.041	0.172 ± 0.078	0.670 ± 0.098
moderate	rem	0.920 ± 0.041	0.101 ± 0.056	0.814 ± 0.078
moderate	zne	0.910 ± 0.042	0.116 ± 0.044	0.801 ± 0.068
moderate	zne_rem	0.910 ± 0.042	0.070 ± 0.015	0.976 ± 0.047
high	none	0.900 ± 0.056	0.301 ± 0.117	0.411 ± 0.128
high	rem	0.902 ± 0.055	0.222 ± 0.105	0.575 ± 0.133
high	zne	0.905 ± 0.046	0.214 ± 0.107	0.603 ± 0.144
high	zne_rem	0.902 ± 0.050	0.121 ± 0.069	0.847 ± 0.130
exact (noiseless)	VQC	0.920 ± 0.041	0 (reference)	1 (reference)
–	Logistic Regression	0.925 ± 0.049	–	–
–	SVM (RBF)	0.915 ± 0.034	–	–

5.3 Graphs and figures

Figure 12 shows the error of every method for each (n, L) at N0–N2 (the N3 stress level is reported separately, in Tables 2 and 4 and results/summary/table_error.md). At N2, every mitigated method is below the baseline in every (n, L) cell; at N0, the ZNE-based methods are worse than the baseline in every cell. Pooled over all circuits, the unmitigated error rises from 0.021 (N0) to 0.062 (N1), 0.181 (N2) and 0.310 (N3). Figure 13 shows the error against qubit count at N2 for each depth, and Figure 14 gives the median improvement for every (n, L) cell at N1 and N2.

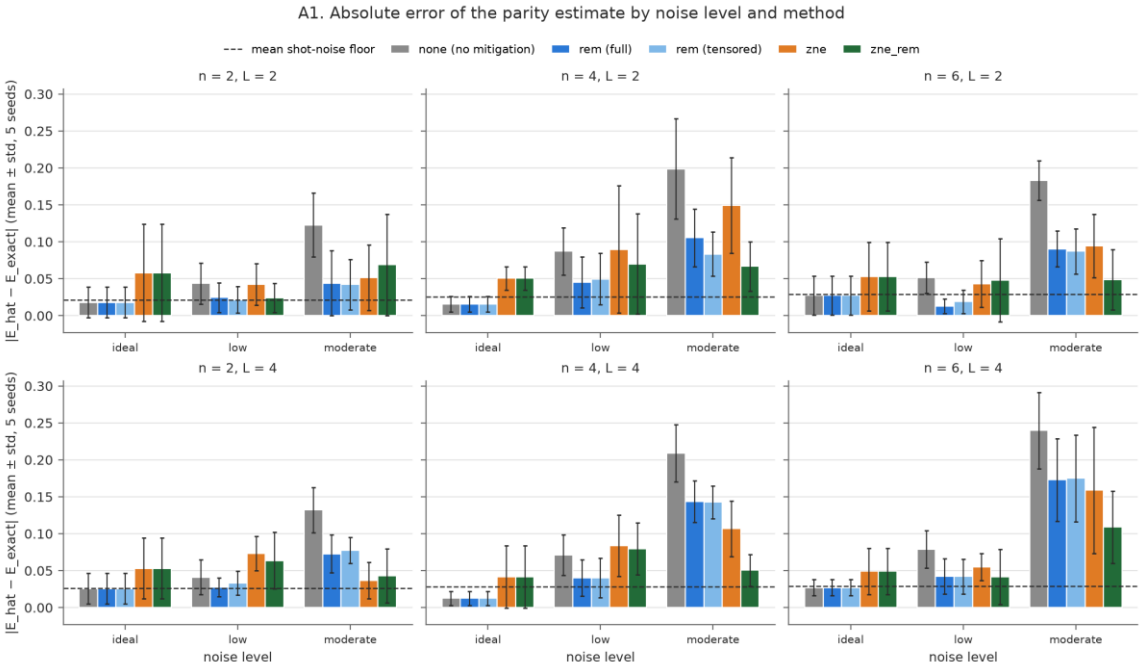


Figure 12: Absolute parity error (mean ± SD over 5 seeds) by noise level (N0–N2) and method, one panel per (n, L); dashed line: mean shot-noise floor.

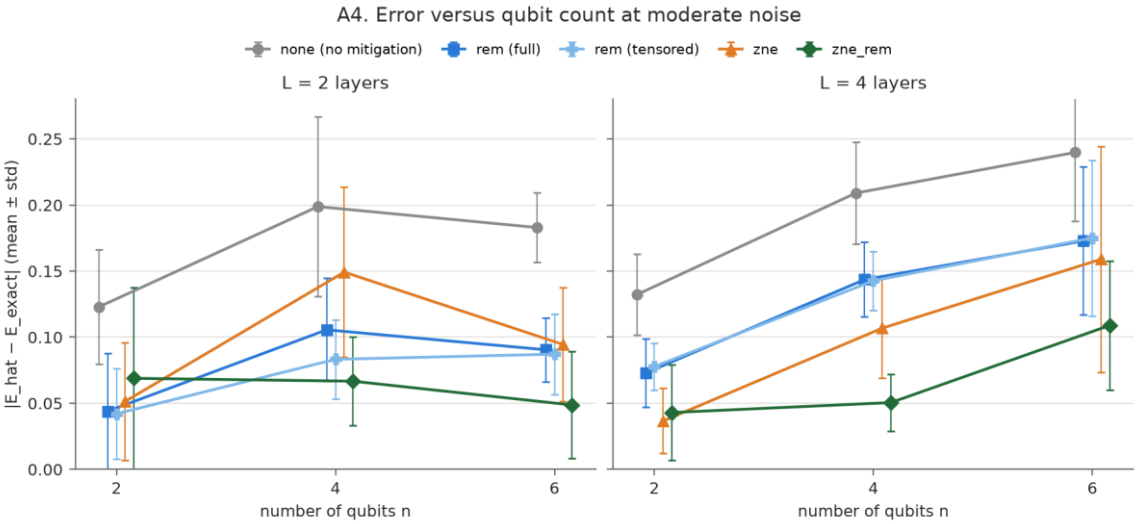


Figure 13: Absolute error vs. qubit count at N2, one panel per depth (mean ± SD over 5 seeds).

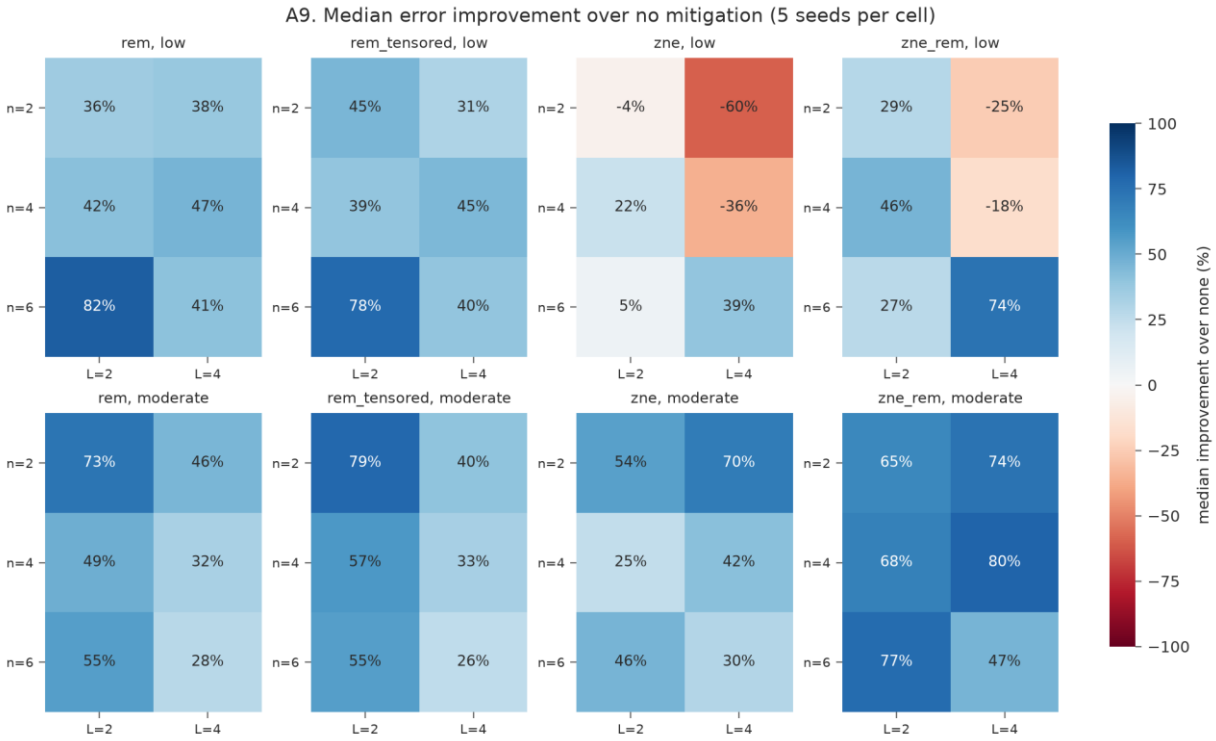


Figure 14: Median improvement over no mitigation (%) for every (n, L) cell at N1 and N2 (5 seeds per cell).

Figure 15 compares the measured output distribution with the exact one for one circuit (n = 4, L = 4, N2, seed 0): noise spreads probability away from the most likely outcomes, and REM (full and tensored) moves it back. Figures 16 and 17 quantify this over the grid: at N2, REM raises Hellinger fidelity in 6/6 (n, L) cells (mean gain 0.0160) and raises the success-probability ratio toward 1.

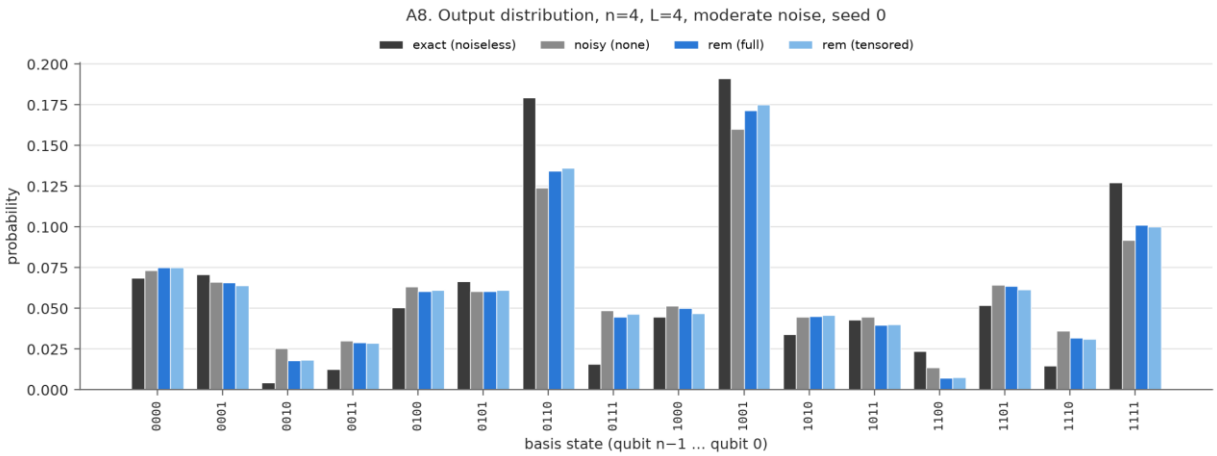


Figure 15: Output distribution for n = 4, L = 4, N2, seed 0: exact, unmitigated, full REM and tensored REM.

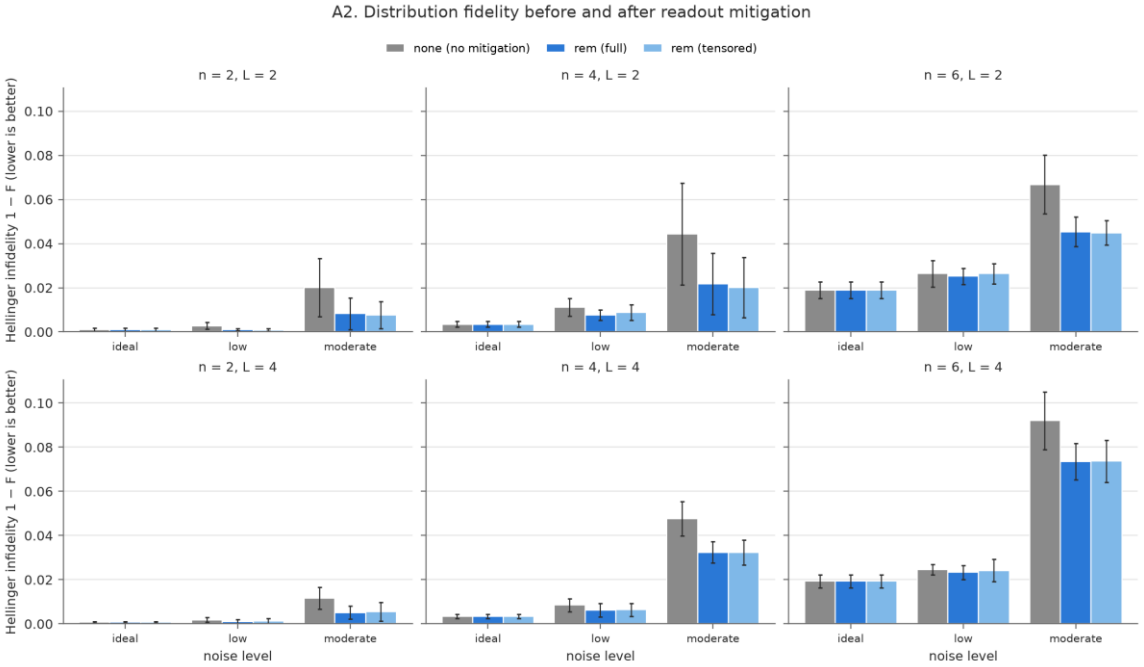


Figure 16: Hellinger infidelity (1 − F) by noise level (N0–N2): no mitigation vs. full and tensored REM, one panel per (n, L).

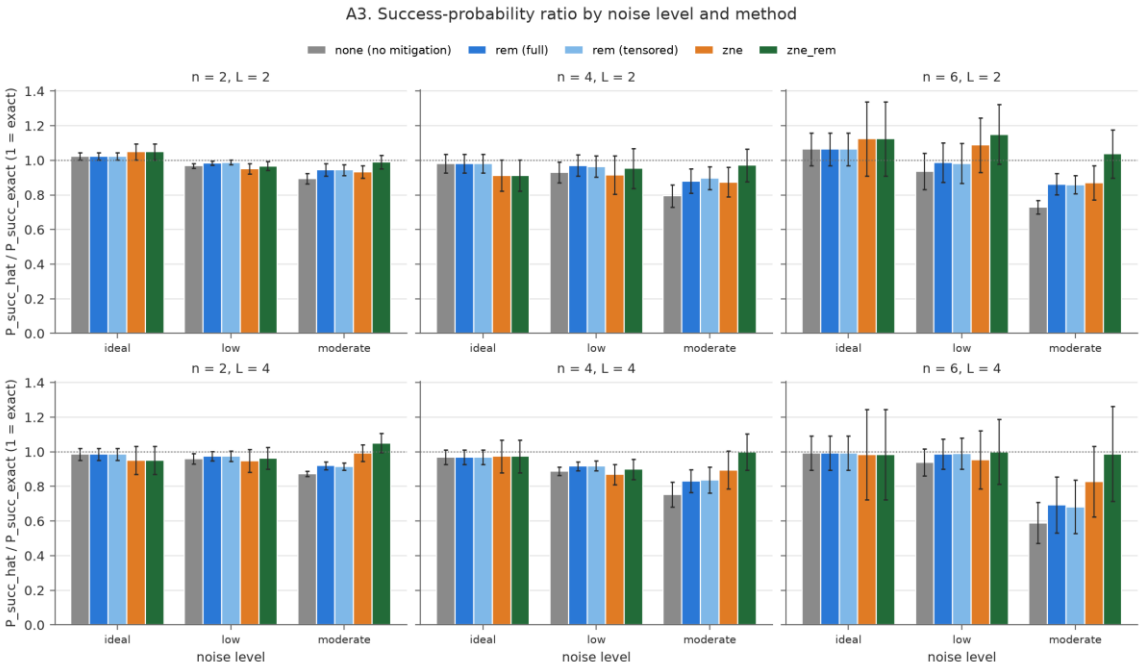


Figure 17: Success-probability ratio P_succ_hat / P_succ_exact (1 = exact) by noise level (N0–N2) and method, one panel per (n, L).

Figure 18 shows the ZNE extrapolation for one circuit (n = 4, L = 4, N2, seed 0; exact parity 0.408, unmitigated 0.197). Extrapolating the raw points (ZNE) gives 0.256, still short of the exact value, partly because readout error is not folded; extrapolating the REM-corrected points (ZNE+REM) gives 0.325, closer but not exact. Figure 19 shows the bias–spread trade-off at N2, pooled over all circuits: unmitigated estimates are shrunk toward 0 (median oriented error −0.183); REM, tensored REM and ZNE each remove about half of that shrinkage (medians −0.093, −0.091, −0.098); only ZNE+REM is centred near zero (+0.015), and the ZNE-based methods have the widest spread (SD 0.072 and 0.080, against 0.059 with no mitigation).

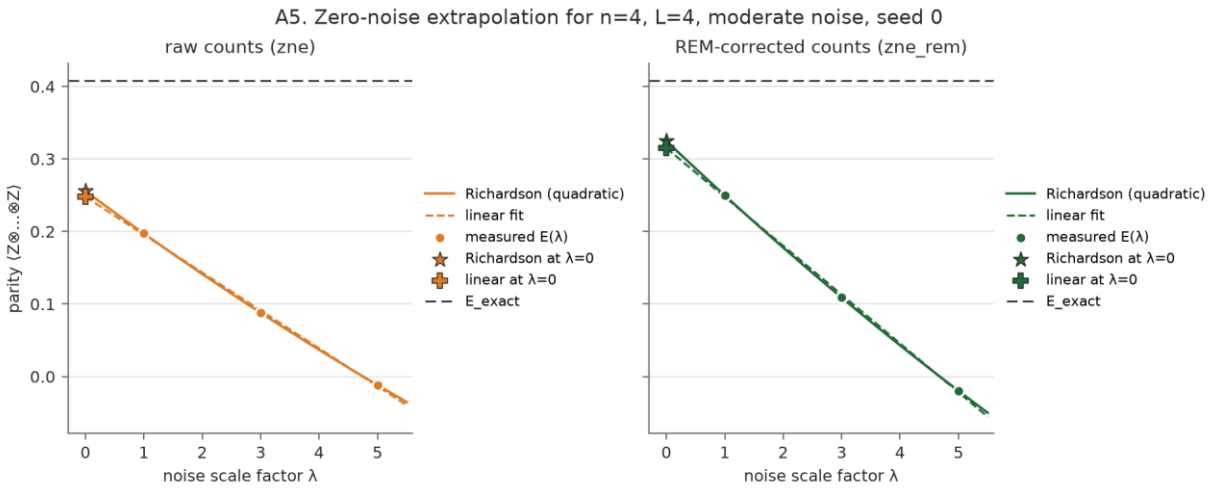


Figure 18: ZNE extrapolation for n = 4, L = 4, N2, seed 0: raw counts (ZNE) vs. REM-corrected counts (ZNE+REM).

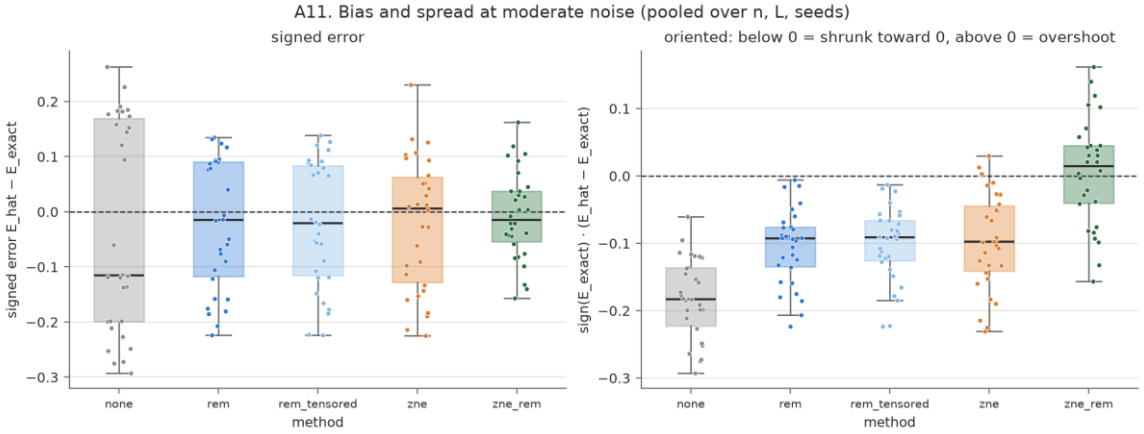


Figure 19: Bias and spread at N2, pooled over n, L and seeds: signed error (left) and error oriented so that negative means shrunk toward 0 (right).

For Track B, Figure 20 shows that VQC accuracy barely moves with noise (N0–N3) and stays close to Logistic Regression and SVM. Figures 21 and 22 show what accuracy hides: the mean expectation error grows with noise, and for n = 4, L = 2 the outputs shrink toward 0 at N2 and N3, mostly without changing sign, while ZNE+REM pulls them back toward the diagonal. Figure 23 shows the noiseless training curves.

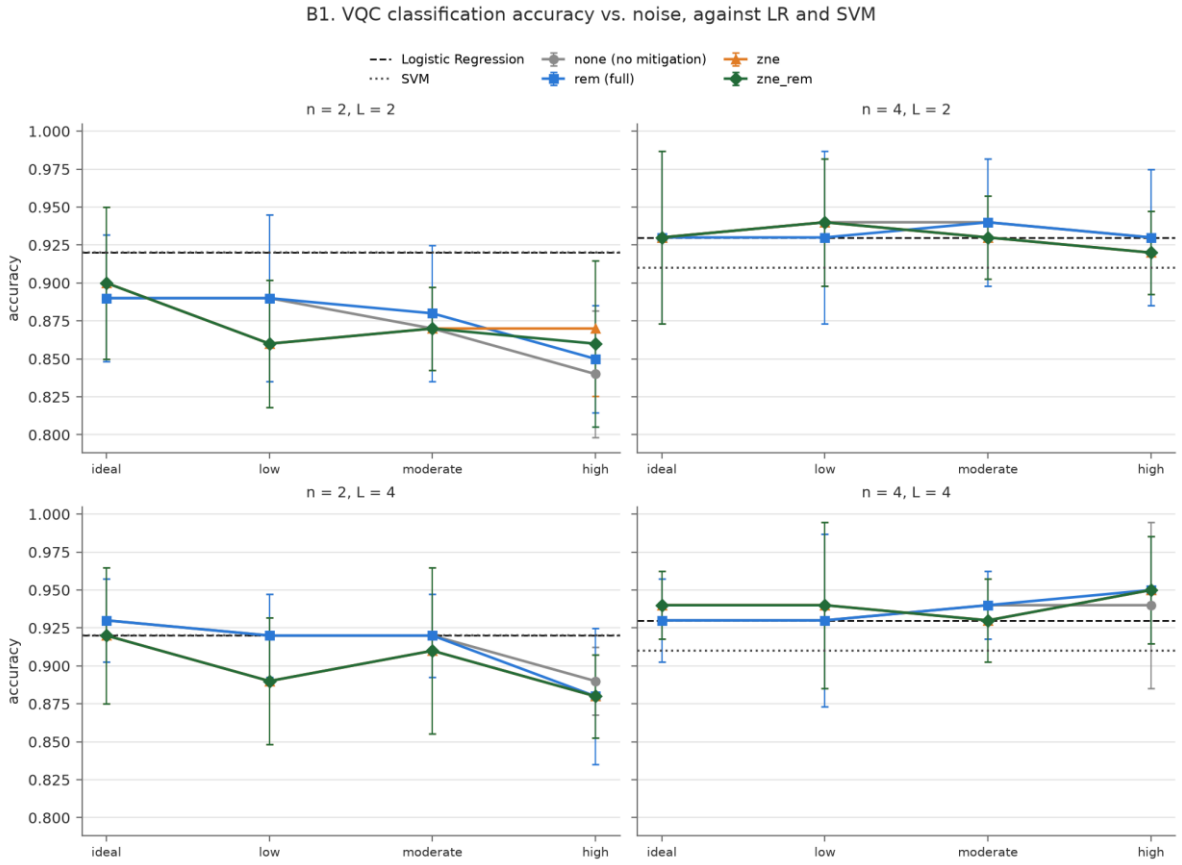


Figure 20: VQC test accuracy vs. noise level (N0–N3) by method, one panel per (n, L) (mean ± SD over 5 seeds); Logistic Regression and SVM as horizontal lines.

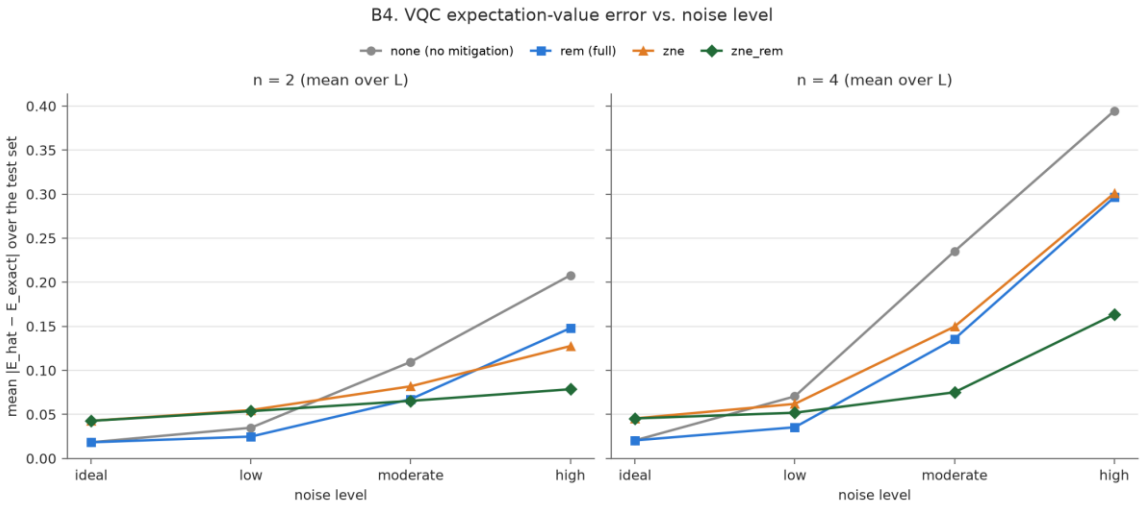


Figure 21: Mean |E_hat − E_exact| over the VQC test set vs. noise level, for n = 2 and n = 4 (mean over L).

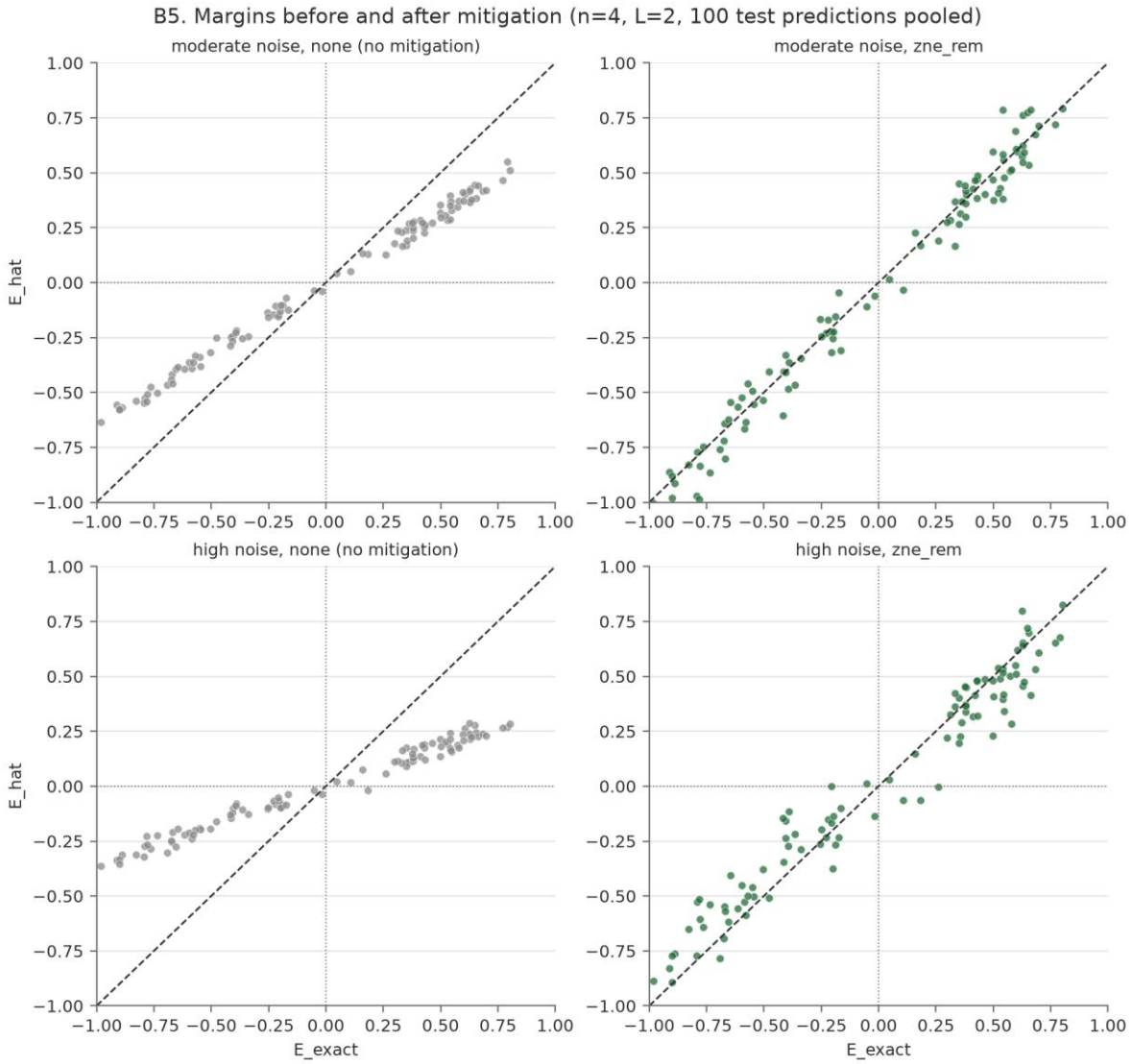


Figure 22: E_hat against E_exact for n = 4, L = 2 (100 test predictions pooled over 5 seeds) at N2 and N3, without mitigation and with ZNE+REM; the diagonal is E_hat = E_exact.

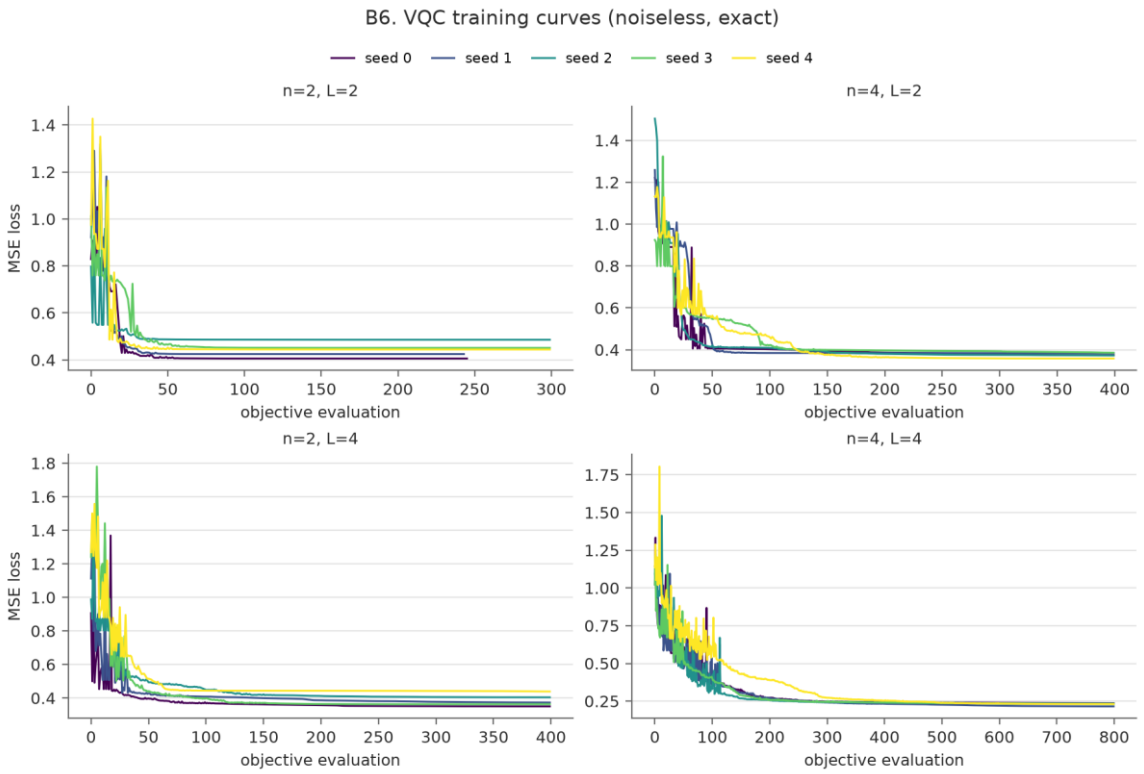


Figure 23: VQC training curves (noiseless): MSE loss against COBYLA objective evaluations, per (n, L) and seed.

Figure 24 compares tensored and full REM condition by condition (points on the diagonal agree). Figure 25 compares the empirical standard deviation of the Richardson estimate under uniform and variance-optimal shot allocation at the same 3072-shot budget, averaged over the 5 circuit instances of each condition: the optimal split is lower in 12 of 12 conditions.

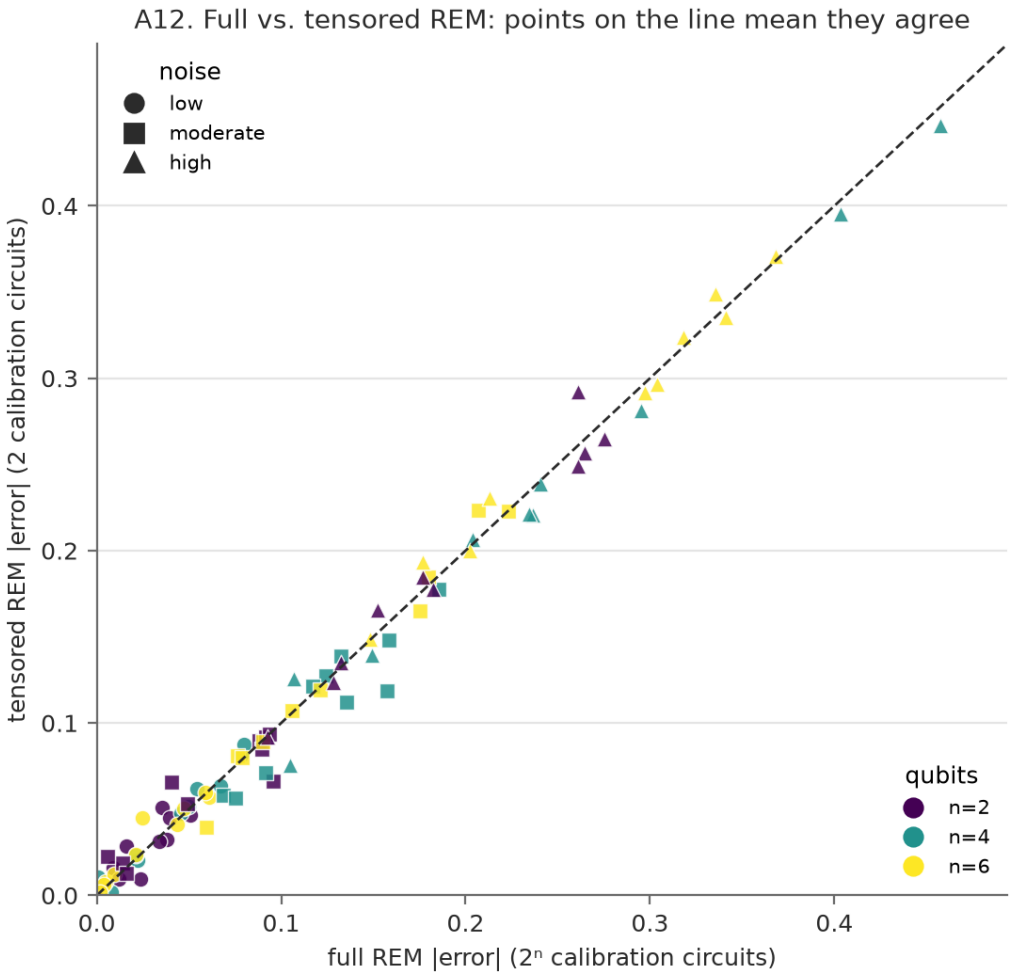


Figure 24: Full vs. tensored REM absolute error for every condition at N1–N3 (marker: noise level; colour: qubit count).

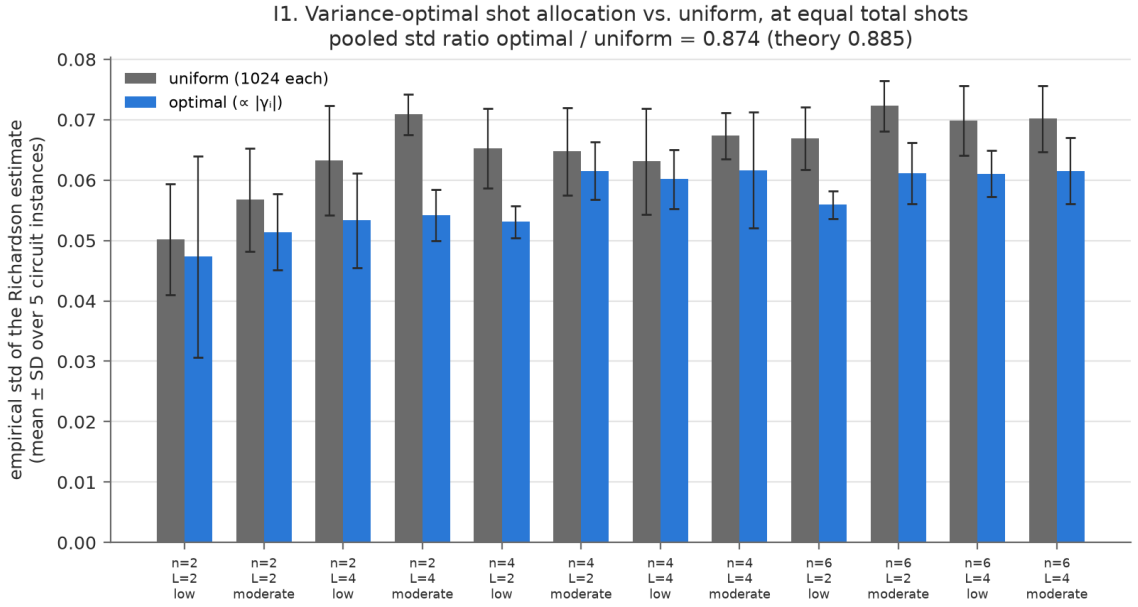


Figure 25: Empirical std of the Richardson estimate, uniform vs. optimal allocation at 3072 total shots (mean ± SD over the 5 circuit instances of each condition).

6. Comparative Analysis (2 marks)

6.1 Comparison with baseline

Table 4. Methods against the no-mitigation baseline (Track A pooled over n, L and 5 seeds: 30 runs per noise level) and the VQC against the classical baselines (identical splits and features).

Model / method	Primary metric (mean ± SD)	Resources (qubits / depth / gates)	Remarks
No mitigation (baseline)	|error| N2: 0.181 ± 0.059 N1: 0.062 ± 0.030	1 circuit; base depth 6–19; 2–20 CNOTs	Error grows 0.021 → 0.062 → 0.181 → 0.310 from N0 to N3.
REM (full)	N2: 0.105 ± 0.056 N1: 0.032 ± 0.024	5/17/65 circuits (n = 2/4/6); depth and CNOTs unchanged	Median cut 43% at N2 (p = 1.9e-09), 42% at N1 (p = 8.0e-08).
REM (tensored)	N2: 0.101 ± 0.055 N1: 0.034 ± 0.024	3/3/3 circuits; depth and CNOTs unchanged	Median cut 48% at N2 (p = 1.9e-09); within 0.0063 of full REM.
ZNE (Richardson)	N2: 0.100 ± 0.067 N1: 0.065 ± 0.045	3/3/3 circuits; 9× CNOTs; up to 5.00× depth	Median cut 44% at N2 (p = 3.7e-09); not significant at N1 (1%, p = 0.984).
ZNE + REM	N2: 0.064 ± 0.046 N1: 0.055 ± 0.045	7/19/67 circuits; 9× CNOTs; up to 5.00× depth	Best at N2: median cut 68% (p = 4.7e-08), lowest error in 4/6 cells.
VQC, exact (noiseless)	Accuracy 0.920 ± 0.041	2–4 qubits; 8–32 parameters; 2–12 CNOTs	Comparable to LR and SVM; no quantum advantage is claimed.
Logistic Regression	Accuracy 0.925 ± 0.049	Classical (C = 1, max_iter = 1000)	Same split, same preprocessed features.
SVM (RBF)	Accuracy 0.915 ± 0.034	Classical (C = 1, gamma = "scale")	Same split, same preprocessed features.

6.2 Effect of noise and resources

Effectiveness (RQ1). At N2 every method reduces the error (Table 4, Figure 12): median cuts of 43% (REM), 48% (tensored REM), 44% (ZNE) and 68% (ZNE+REM), all significant in pooled Wilcoxon tests. At N1 the picture changes: REM still helps (42%, p = 8.0e-08), but ZNE alone shows no significant benefit (1%, p = 0.984) because its variance amplification outweighs a small gate bias. At the N3 stress level every method still helps (29%, 25%, 19%, 58%) and ZNE+REM stays strongest.

Why the pattern (readout vs. gate noise). Splitting the unmitigated bias exactly with the noisy density matrix shows that readout causes 60% of it at N1 but only 40% at N2, as gate noise grows with circuit size. REM fixes only readout and ZNE only gate noise, so REM matters more at N1, ZNE more at N2, and only the combination addresses both (H2, H3).

Variance. ZNE's standard deviation is 2.38×, 2.42×, 2.70× and 2.07× the baseline's at N0–N3, against 2.28× predicted by the Richardson weights. Each ratio compares two standard deviations estimated from 30 runs, which carries roughly ±19% relative uncertainty, so these values are consistent with theory. At N0 there is no bias to remove, and ZNE was worse than no mitigation in 26/30 runs (Figure 12).

Cost (RQ2). Per estimate, full REM needs $1 + 2^n$ circuits (5/17/65 at n = 2, 4, 6) while tensored REM needs a constant 3; ZNE needs 3 circuits but 9× the CNOTs and up to 5.00× the depth (95 at the deepest); ZNE+REM needs the sum (Figures 26 and 27). Simulator wall time is not hardware time, so circuit and shot counts are the meaningful cost figures.

Downstream classification (RQ3). The exact VQC reaches 0.920 mean accuracy, comparable to Logistic Regression (0.920 / 0.930 for n = 2 / 4) and SVM (0.920 / 0.910) on identical inputs (Tables 3 and 4). At N3, unmitigated margin retention falls to 0.411 and mean expectation error rises to 0.301, yet accuracy only drops from 0.920 to 0.900; ZNE+REM restores margin retention to 0.847 with almost no change in accuracy (0.903). Accuracy depends only on the sign of the output and moves in 5% steps on 20 test samples, so it is too coarse to show what the continuous metrics show.

Cheaper variants (RQ4). Tensored REM matches full REM to within 0.0063 on average (worst 0.0388) at 3 circuits instead of up to 65, and variance-optimal shot allocation (1646,1097,329 shots at λ = 1, 3, 5) gives a

standard deviation 0.874× the uniform split's, against 0.885× predicted. No quantum advantage is claimed anywhere.

Table 5. Hypotheses and verdicts.

	Hypothesis	Verdict	Evidence
H1	Unmitigated error grows with noise level, n and L.	Partly supported	Rises with noise level in all 6 (n, L) cells (pooled 0.021 → 0.062 → 0.181 → 0.310). Averaged over the other parameter, error is lowest at n = 2 and higher at L = 4 than L = 2 at N1–N3, but not in every cell: at N2, n = 4, L = 2 has 0.199 vs 0.183 for n = 6, L = 2, because each cell uses different circuit instances (Figures 12 and 13).
H2	REM removes most of the readout bias, improves fidelity and success probability, and leaves the gate-noise bias.	Supported	REM's remaining error at N2 (0.105) is close to the gate part of the bias (0.111); fidelity and P_succ improve (Figures 16 and 17).
H3	ZNE removes much of the gate-noise bias but not the readout bias, because measurements are not folded.	Supported	No significant benefit at N1, where readout is 60% of the bias; a large one at N2 (readout share 40%).
H4	ZNE+REM is best at N1 and N2; ZNE degrades at N3.	Partly supported	Best at N2 (4/6 cells) and N3 (median 58%); at N1 plain REM wins more often (4/6 vs 1/6). No runaway failure at N3 (Richardson out of range 0/120), but more exponential fits fail.
H5	ZNE has higher variance than no mitigation; at N0 it is no better.	Supported	Std ratio 2.07×–2.70× (theory 2.28×, within the ±19% sampling uncertainty of 30 runs); worse than no mitigation at N0 in 26/30 runs.
H6	REM uses $1+2^n$ circuits, tensored REM 3, ZNE 3 with 9× CNOTs and ≈5× depth, ZNE+REM the sum.	Supported	Measured 5/17/65, 3/3/3, 3/3/3 and 7/19/67; 9× CNOTs, 5.00× depth (Figure 26).
H7	Exact VQC accuracy is comparable to or below LR and SVM.	Supported	VQC 0.920 vs LR 0.920/0.930 and SVM 0.920/0.910 (n = 2 / 4).
H8	Noise shrinks margins but keeps their sign; mitigation restores margins more than accuracy.	Supported	N3: margin retention 0.411 → 0.847 with ZNE+REM, accuracy 0.900 → 0.903 (Figures 21 and 22).
H9	Tensored REM matches full REM within shot noise.	Supported	Mean |difference| 0.0063, max 0.0388 (Figure 24).
H10	Optimal shot allocation cuts ZNE's std by about 11% at equal budget.	Supported	Measured ratio 0.874 vs 0.885 predicted (Figure 25).

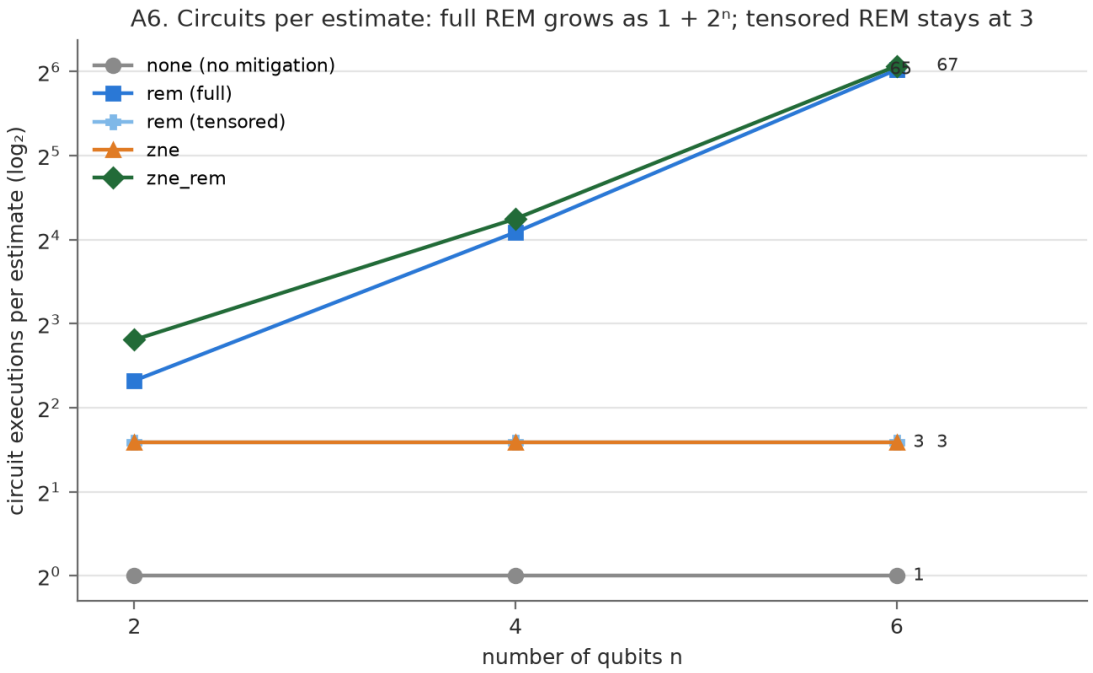


Figure 26: Circuits per estimate vs. qubit count: full REM grows as 1 + 2ⁿ, tensored REM stays at 3.

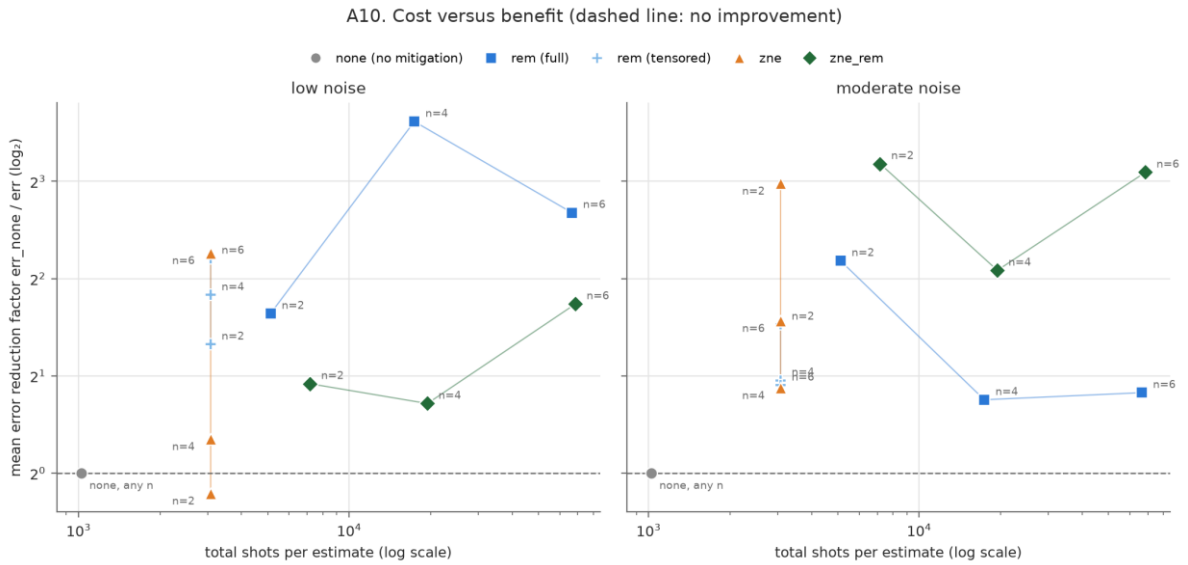


Figure 27: Cost (total shots per estimate) against benefit (mean error reduction factor), both on log scales, at N1 and N2; dashed line: no improvement.

7. Research Gap, Limitations and Proposed Improvement (2 marks)

7.1 Limitations

- **Simulation only.** The depolarizing-plus-readout model is incoherent and memoryless; real devices add coherent errors, crosstalk, T1/T2 relaxation, drift and leakage.
- **ZNE.** It amplifies variance (measured 2.07×–2.70×, consistent within sampling uncertainty with the 2.28× theory), its extrapolation model can mismatch the true decay (the exponential fit failed in 10/120 ZNE conditions), it never removes readout error, and its deepest folded circuit (95 deep, 5× the base) could exceed coherence times on hardware.
- **REM.** Full calibration costs 2^n circuits; REM assumes stable readout; it can produce negative quasi-probabilities (largest negative mass 0.0186); it does nothing for gate noise. Tensored REM also assumes uncorrelated readout, which holds by construction in our noise model, so the comparison favours it.

- **Small samples.** Three qubit counts, two depths and 5 seeds per condition limit statistical power; with 20 test samples per seed, accuracy moves in 5% steps.
- **Selection bias.** Track A keeps only circuits with |E_exact| ≥ 0.3, so near-zero-parity circuits are not covered.
- **VQC.** Trained noise-free with a single encoding layer, which limits expressivity [15]; noise-aware training was not attempted [16].
- **Scale and time.** At most 6 qubits; simulator wall time is not hardware execution time.

7.2 Proposed improvement / future direction

Tested in this study. (1) Tensored REM: 2 calibration circuits instead of 2^n, matching full REM within 0.0063 on average. (2) Variance-optimal ZNE shot allocation: minimizing $\mathrm{Var}(E_o) = \Sigma_i \gamma_i^2 \sigma_i^2 / N_i$ subject to $\Sigma N_i = N$ gives $N_i \propto |\gamma_i| \sigma_i$, so 3072 shots split as 1646,1097,329 instead of 1024 each; the measured standard deviation fell to 0.874× the uniform split (theory 0.885×) at the same total cost.

Proposed next step: validate on real hardware with correlated readout. Run the same paired protocol (same circuits, same shots shared across methods) on an IBM Quantum device, comparing full REM, tensored REM and matrix-free M3 mitigation [8], with and without ZNE. Why it should help: our results favour tensored REM partly because simulated readout errors are independent by construction. Real devices have correlated readout errors and drift, so a hardware run would show whether the 2-circuit calibration still matches the 2^n one, and whether M3 recovers the gap at larger qubit counts where full calibration is impossible. The same run would test whether folding scales real, partly coherent noise as cleanly as it scales our depolarizing noise.

Further directions: data re-uploading to lift the VQC's Fourier-frequency limit [15], noise-aware VQC training with the caveats of [16], and probabilistic error cancellation or Clifford data regression on a smaller, targeted grid.

8. Key Findings Summary for Journal Paper

Top 3 findings (with numbers)	1. At N2, ZNE+REM cut the median parity error by 68% (REM 43%, tensored REM 48%, ZNE 44%; all pooled Wilcoxon p ≤ 4.7e-08); at N1, ZNE alone gave no significant benefit (1%, p = 0.984) while REM gave 42%. 2. Tensored REM matched full REM within 0.0063 mean |difference| using 3 circuits instead of up to 65, and variance-optimal shot allocation reduced ZNE's std to 0.874× (theory 0.885×) at an equal 3072-shot budget. 3. On the VQC at N3, ZNE+REM restored margin retention from 0.411 to 0.847 while accuracy barely changed (0.900 vs 0.903); the exact VQC (0.920) is comparable to LR and SVM.
Best configuration (qubits / depth / noise) and why	Largest median error cut overall: ZNE+REM at N3, n = 4, L = 2 (87%), at the stress level. At the headline levels: REM at N1, n = 6, L = 2 (82%), error 0.051 ± 0.021 → 0.013 ± 0.010; and ZNE+REM at N2, n = 4, L = 4 (80%), error 0.209 ± 0.039 → 0.051 ± 0.021. Why: REM alone wins where readout dominates the bias (readout share 65% in that N1 cell); ZNE+REM wins where gate noise dominates (readout share 29% in that N2 cell). Where cost matters, tensored REM gives the same benefit as full REM (within 0.0063) for 3 circuits at any n.
What is new compared with the reviewed literature?	A paired, cost-normalized side-by-side comparison of full REM, tensored REM, ZNE and ZNE+REM in which every method processes the same shots, carried through to a VQC compared with LR and SVM on identical inputs, plus an empirical test of variance-optimal ZNE shot allocation.
Negative / unexpected results	ZNE alone was not significant at N1 and was worse than no mitigation at N0 in 26/30 runs. Unmitigated error grows with noise in every cell but not monotonically with n and L in every cell (H1 only partly supported). No runaway ZNE failure at N3 (Richardson out of range 0/120), although exponential fits failed more often. Classification accuracy was insensitive to both noise and mitigation.

Figures/tables suitable for the paper (IDs)	Figures 12, 14, 22, 24, 25 and 27; Tables 2, 3, 4 and 5.
Suggested keywords	Quantum error mitigation; zero-noise extrapolation; readout error mitigation; NISQ; variational quantum classifier; Qiskit Aer; depolarizing noise; shot allocation

Conclusion

REM, ZNE and their combination all reduce expectation-value error once gate noise is significant (N2 and N3), and ZNE+REM is the strongest method there, with a median error cut of 68% at N2. At low noise (N1), REM (full or tensored) is the dependable choice, and ZNE alone does not help because its variance cost outweighs a small bias. Costs scale as expected: $1 + 2^n$ circuits for full REM, 3 for tensored REM, 3 circuits with 9× the CNOTs and up to 5× the depth for ZNE, and the sum for ZNE+REM. On the variational classifier, mitigation restores the decision margins strongly but barely moves accuracy, which is too coarse a measure on 20 test samples, and the exact VQC is comparable to, not better than, Logistic Regression and SVM. Both proposed cheaper variants worked: tensored REM matched full REM at a fraction of the circuits, and variance-optimal shot allocation reduced ZNE's standard deviation close to the predicted amount at the same budget.

Recommendation. Use REM by default (tensored once full calibration becomes impractical). Add ZNE, preferably combined with REM, when gate noise is large relative to shot noise and the extra depth and shots are affordable; skip ZNE when noise is already low. For classification tasks, judge mitigation by expectation error and margins, not by accuracy alone.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018, doi: 10.22331/q-2018-08-06-79.

[2] K. Temme, S. Bravyi, and J. M. Gambetta, "Error mitigation for short-depth quantum circuits," Phys. Rev. Lett., vol. 119, no. 18, p. 180509, 2017, doi: 10.1103/PhysRevLett.119.180509.

[3] Y. Li and S. C. Benjamin, "Efficient variational quantum simulator incorporating active error minimization," Phys. Rev. X, vol. 7, no. 2, p. 021050, 2017, doi: 10.1103/PhysRevX.7.021050.

[4] S. Endo, S. C. Benjamin, and Y. Li, "Practical quantum error mitigation for near-future applications," Phys. Rev. X, vol. 8, no. 3, p. 031027, 2018, doi: 10.1103/PhysRevX.8.031027.

[5] A. Kandala, K. Temme, A. D. Córcoles, A. Mezzacapo, J. M. Chow, and J. M. Gambetta, "Error mitigation extends the computational reach of a noisy quantum processor," Nature, vol. 567, pp. 491–495, 2019, doi: 10.1038/s41586-019-1040-7.

[6] T. Giurgica-Tiron, Y. Hindy, R. LaRose, A. Mari, and W. J. Zeng, "Digital zero noise extrapolation for quantum error mitigation," in Proc. IEEE Int. Conf. Quantum Computing and Engineering (QCE), 2020, pp. 306–316, doi: 10.1109/QCE49297.2020.00045.

[7] S. Bravyi, S. Sheldon, A. Kandala, D. C. McKay, and J. M. Gambetta, "Mitigating measurement errors in multiqubit experiments," Phys. Rev. A, vol. 103, no. 4, p. 042605, 2021, doi: 10.1103/PhysRevA.103.042605.

[8] P. D. Nation, H. Kang, N. Sundaresan, and J. M. Gambetta, "Scalable mitigation of measurement errors on quantum computers," PRX Quantum, vol. 2, no. 4, p. 040326, 2021, doi: 10.1103/PRXQuantum.2.040326.

[9] R. LaRose et al., "Mitiq: A software package for error mitigation on noisy quantum computers," Quantum, vol. 6, p. 774, 2022, doi: 10.22331/q-2022-08-11-774.

[10] Y. Kim et al., "Evidence for the utility of quantum computing before fault tolerance," Nature, vol. 618, pp. 500–505, 2023, doi: 10.1038/s41586-023-06096-3.

[11] R. Takagi, S. Endo, S. Minagawa, and M. Gu, "Fundamental limits of quantum error mitigation," npj Quantum Inf., vol. 8, p. 114, 2022, doi: 10.1038/s41534-022-00618-z.

[12] Y. Quek, D. Stilck França, S. Khatri, J. J. Meyer, and J. Eisert, "Exponentially tighter bounds on limitations of quantum error mitigation," Nat. Phys., vol. 20, pp. 1648–1658, 2024, doi: 10.1038/s41567-024-02536-7.

[13] Z. Cai et al., "Quantum error mitigation," Rev. Mod. Phys., vol. 95, no. 4, p. 045005, 2023, doi: 10.1103/RevModPhys.95.045005.

[14] V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," Nature, vol. 567, pp. 209–212, 2019, doi: 10.1038/s41586-019-0980-2.

[15] M. Schuld, R. Sweke, and J. J. Meyer, "Effect of data encoding on the expressive power of variational quantum-machine-learning models," Phys. Rev. A, vol. 103, no. 3, p. 032430, 2021, doi: 10.1103/PhysRevA.103.032430.

[16] S. Wang, E. Fontana, M. Cerezo, K. Sharma, A. Sone, L. Cincio, and P. J. Coles, "Noise-induced barren plateaus in variational quantum algorithms," Nat. Commun., vol. 12, p. 6961, 2021, doi: 10.1038/s41467-021-27045-6.

[17] J. A. Smolin, J. M. Gambetta, and G. Smith, "Efficient method for computing the maximum-likelihood quantum state from measurements with additive Gaussian noise," Phys. Rev. Lett., vol. 108, no. 7, p. 070502, 2012, doi: 10.1103/PhysRevLett.108.070502.

[18] R. A. Fisher, "The use of multiple measurements in taxonomic problems," Ann. Eugenics, vol. 7, no. 2, pp. 179–188, 1936, doi: 10.1111/j.1469-1809.1936.tb02137.x.

[19] R. A. Fisher, "Iris," UCI Machine Learning Repository, 1936, doi: 10.24432/C56C76.

[20] A. Javadi-Abhari et al., "Quantum computing with Qiskit," arXiv:2405.08810, 2024, doi: 10.48550/arXiv.2405.08810.

[21] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," Zenodo, doi: 10.5281/zenodo.2573505. Version used: qiskit 2.5.2.

[22] Qiskit Aer contributors, "Qiskit Aer: high-performance simulators for Qiskit," version 0.17.2. [Online]. Available: https://github.com/Qiskit/qiskit-aer

[23] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011. Version used: 1.9.1.

Appendix A – Contribution Statement

Student	Specific contribution (coding / experiments / writing / analysis)
G Srikaanth (1MS23CY015): about 70%	Coding: circuit construction, noise model and executor, full and tensored REM, ZNE (folding and extrapolation), the Track A sweep, the VQC and the shot-allocation experiment. Experiments: ran all Track A, Track B and shot-allocation sweeps. Analysis: statistical tests, figures and cost analysis. Writing: Sections 1 and 3–8 of the report, the results dashboard and the study guide.
Devansh Sharma (1MS23CY014): about 30%	Coding: Iris data loading and preprocessing, the Logistic Regression and SVM baselines, and their unit tests. Experiments: checked the Track B runs and classification metrics. Writing: the literature review (Section 2), limitations and references; proofreading and viva preparation.

Appendix B – Declaration

We declare that this work is our own, that all sources are cited, that experiments were actually run and results are not fabricated, and that any use of AI tools (including code assistants) is disclosed below. We consent to the use of this work, with acknowledgement, in a research paper prepared by the faculty guide.

AI tools used (name & purpose)	Claude Code (Anthropic), an AI coding assistant: used to help write and debug the Python code, run the experiment scripts, and draft the report text and documentation. Every reported number is produced by the project's code from the actual runs (results/ in the repository).
Signature of team members	G Srikaanth: _____ Devansh Sharma: _____

Appendix C – Submission Checklist

- ☑ Report in PDF format (file name: Group20_Quantum_Report.pdf)
- ☑ Code runs end-to-end; repository / Colab link works for the faculty
- ☑ Circuit diagrams and screenshots are legible
- ☑ At least 6 controlled experiments with 1024 shots and ≥2 varied parameters

- ☑ Mean ± SD over 5 seeds where feasible; noise levels N0–N2 reported
- ☑ At least 5 papers reviewed and a research gap stated
- ☑ Limitations and a proposed improvement included
- ☐ Google Form submitted (one submission per group, by the group leader)

Evaluation (for faculty use)

Component	Marks	Awarded / remarks
Problem understanding and objectives	2	
Literature review (min. 5 papers, research gap)	3	
Algorithm / circuit design	3	
Qiskit implementation	4	
Experiments and visualization	3	
Comparative analysis	2	
Research gap and proposed improvement	2	
Report clarity and response to questions	1	
Total	20	